

Learning-Rate Annealing Sets the Findable Capacity of the Tempotron

Computational & Theoretical SNN Research Workspace

June 14, 2026

Abstract

The tempotron classifies spike patterns on the *maximum over time* of its membrane voltage. Rubin, Monasson & Sompolinsky (RMS, 2010) showed that this maximum-over-time lifts the storage capacity above the perceptron’s $\alpha_c = 2$ and shatters the solution space into many disconnected clusters — separating the *existence* of zero-error weights from their *findability* by a gradient rule. On the Gütig–Sompolinsky (GS) single-spike latency ensemble, the GS error-correcting rule with a small constant step reaches a findable $\alpha_c \approx 2.5$, well below the $\alpha_c \approx 3$ GS *report*. We show that this shortfall is, on this ensemble and at the scale tested, *not* a property of the temporal classifier but of an under-optimised learning rule, and that the single responsible ingredient is the *annealing* of the learning rate toward zero. To establish this we first build a measurement pipeline whose every component — the postsynaptic-potential kernel, the random spike-pattern ensemble, the decision rule and its threshold calibration, and the learning rule — is verified against its closed form, and we calibrate the end-to-end procedure against known answers: the exact perceptron capacity $\alpha_c = 2$ and the Gardner margin ($N \leq 500$, two independent separability oracles), the Gumbel extreme-value statistics of the peak voltage, and the kernel. As a falsification control we remove spike timing (the rate-coding limit): capacity returns to the perceptron value $\alpha_c \approx 2$ — by both a trainer-independent existence oracle (2.02) and the GS-constant- λ rule ($\rightarrow 2$) — confirming that the single-spike lift is a property of the temporal code, not of our pipeline. A pre-registered optimiser study and a controlled one-factor schedule experiment then show that a fairly tuned, annealed learner is far more *findable* than the matched-peak constant-schedule rule, which collapses by $\alpha \approx 2.5$, and that the single responsible ingredient is the anneal — not the optimiser family or the peak step. We are deliberately conservative about how far this reaches. Across committed $N = 500$ runs the annealed learner attains exact zero error reliably through $\alpha = 2.8$, but its behaviour at $\alpha = 3.0$ is strongly *configuration-sensitive*: nominally similar Adam–cosine runs differing only in seed count, warm-up, and time grid solve anywhere from 2/24 to 16/16 tasks, and several solve curves are bootstrap-unpinned, with half-crossings spanning ≈ 2.9 (24-seed run) to > 3.1 . We therefore report a schedule- and optimiser-dependent improvement in *finite-budget findability* — *not* a robust constructive capacity at $\alpha = 3$. The matched-budget findable crossing rises with both training time and N ($\alpha_c \approx 2.7$ for $N \leq 250$), but the $N \geq 500$ behaviour is not pinned at the present seed counts and budget; the bare-rule confirmation uses Adam (the online rule is wall-prohibitive at $N = 500$). A trainer-independent Gardner oracle — which would certify existence independently of the optimiser — remains open, and a faithful reproduction of the GS Supplementary-Methods protocol (initial efficacies, voltage/ $t_{\max}$ convention) is not yet implemented; we therefore label our constant-step rule a *GS-inspired* baseline (§9).

Contents

1 Introduction

3

2	The tempotron model	5
3	The postsynaptic-potential kernel and forward pass	6
4	The random spike-pattern ensemble	7
5	Decision rule and threshold calibration	8
5.1	Convexity and the shattered solution space	9
5.2	Extreme-value theory and the threshold scale	9
5.3	Internal representations and the capacity	9
6	The learning rule	10
6.1	The rule is gradient descent	10
6.2	A subgradient on a piecewise-smooth surface	11
6.3	Existence versus findability	11
6.4	Momentum and the effective step	11
7	Validation of the capacity-measurement methodology	11
7.1	Perceptron capacity	11
7.2	Extreme-value statistics of the decision variable	14
7.3	The postsynaptic kernel	16
7.4	Summary	17
8	Tempotron memory capacity and the finite-size masking of the temporal lift	17
9	The original Gütig–Sompolinsky benchmark: capacity of the single-spike latency ensemble	18
9.1	Why the latency capacity exceeds the perceptron value	19
9.2	The GS-inspired constant- λ protocol	19
9.3	Results	20
10	Findability, solution geometry, and noise robustness	24
11	The threshold–weight-magnitude relationship	25
12	Findable capacity under tuned learning	26
12.1	A pre-registered, fair optimiser comparison	27
12.2	Training procedure: one objective, three optimisers, a tuned batch	28
12.3	Result at the tuning scale ($N = 200$): what makes the temporal classifier learnable	29
12.4	Confirmation at the Gütig–Sompolinsky scale: findability improves, but $\alpha = 3$ is configuration-sensitive	33
13	Which property of the schedule is responsible? A controlled factorial	36
13.1	Background, schedules, and metrics	36
13.2	Design: one factor at a time, at a matched peak	36
13.3	The annealing, not the step size, carries the gain	37
13.4	At the benchmark scale the schedule <i>sets</i> the findable capacity	38
13.5	The exponential shape, and N -independence	40
14	Controls: the rate-coding limit and finite-N scaling	43

1 Introduction

In plain terms. A tempotron is a model neuron that classifies a pattern by the *timing* of the spikes arriving on its input lines, firing if its membrane voltage ever crosses a threshold. Its *capacity* is how many random patterns it can be trained to classify correctly, per input synapse. A standard perceptron tops out at two patterns per synapse; because the tempotron reads out the largest voltage *over time* rather than a single static sum, it should be able to store more — a primitive of interest both for energy-efficient neuromorphic hardware and for understanding temporal coding in the brain. But a higher capacity is only useful if a learning rule can actually *find* the weights that achieve it. This paper asks whether ordinary gradient-based learning reaches the tempotron’s predicted capacity, and finds that it does — once the learning rate is annealed correctly, gradient descent recovers most of the gap to the theoretical bound that an untuned rule appears to miss.

The tempotron [1, 6] is a leaky integrate-and-fire neuron used as a binary classifier of *spatio-temporal* spike patterns. Unlike the perceptron, whose decision is a single linear threshold on a static input vector, the tempotron’s decision is a threshold on the *maximum over time* of a temporally-extended membrane voltage. Rubin, Monasson & Sompolinsky (RMS) [2] showed, via statistical mechanics [17, 19] and extreme-value theory, that this maximum-over-time lifts the storage capacity above the perceptron’s [5] and fractures the solution space into many small clusters.

The question motivating this study concerns the *learning surface*: how and why does a gradient-based learning rule reliably reach a zero-error weight vector when the feasible set is shattered into many disconnected clusters, and is gradient descent the right tool? This separates *existence* — the Gardner volume of solutions, which capacity theory computes — from *findability*, whether the learning dynamics land in it. Put simply: does a correct weight vector *exist*, and if so, can a gradient rule actually *find* it? Probing this empirically requires a simulation pipeline whose every component is independently verified, because a capacity is *inferred* through a measurement procedure that can be biased in ways invisible when the answer is unknown.

Contribution. This paper does two things. First, it builds and calibrates that pipeline against problems with known answers. Second, it uses the calibrated pipeline to show that a GS-inspired learning rule with a small constant step reaches a findable $\alpha_c \approx 2.5$, while a fairly tuned, annealed learner instead reaches exact zero error reliably through $\alpha = 2.8$ at $N = 500$ and improves findability into a configuration-sensitive band near $\alpha \approx 3$ (committed Adam-cosine runs at $\alpha = 3.0$ span 2/24 to 16/16 solved; §12.4). The bare rule’s ≈ 2.5 is thus a *findability* shortfall of an under-optimised learning rule, not a property of the temporal classifier — but we do *not* claim a trainer-independent existence bound at $\alpha \geq 3$. A controlled one-factor study isolates the responsible ingredient: it is the *annealing* of the learning rate toward zero — not the optimiser family or the step size — that carries the learner from ≈ 2.5 to a reliable 2.8 (and, in some configurations, toward ≈ 3). As a falsification control, removing spike timing (the rate-coding limit) returns the capacity toward the perceptron value $\alpha_c \approx 2$ (a control whose committed provenance we are careful about; §14) — evidence that the temporal lift is a property of the code, not an artefact of the pipeline.

Two ensembles. The study uses two random spike-pattern ensembles, and keeping them distinct avoids a common confusion. The *RMS Poisson* ensemble (rate-1/ T spike trains) is the object of

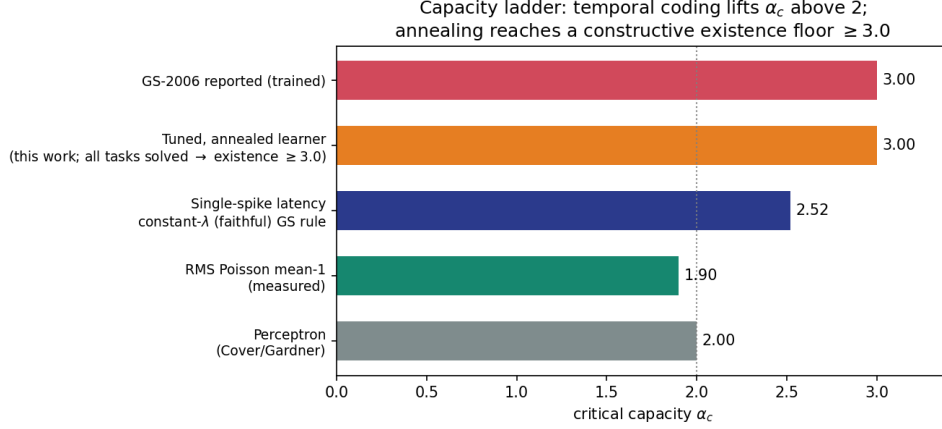


Figure 1: The capacity ladder framing this study: $\alpha_c = 2$ (perceptron) $\rightarrow$ constant-step trained $\approx 2.5 \rightarrow$ annealed trained, reliable through $\alpha = 2.8$ with a configuration-sensitive reach toward ≈ 3 . On the single-spike latency ensemble the bare Gütig–Sompolsky-inspired rule reaches only a findable $\alpha_c \approx 2.5$ at $N = 500$; Sections 9–13 establish each rung and show a tuned, annealed learner solves reliably through $\alpha = 2.8$ (committed runs disagree at $\alpha = 3.0$; §12.4). The findable crossing rises with N (matched-budget $\alpha_c \approx 2.7$ for $N \leq 250$); the existence capacity (a trainer-independent oracle) is left open.

the $\ln \ln K$ capacity-scaling theory and of the finite-size analysis in Section 8; the *GS single-spike latency* ensemble (one spike per afferent, its time the encoded feature) is the celebrated benchmark with a GS capacity $\alpha_c \approx 3$, and is where the findability result lives (Sections 9, 12, 13). Three capacities must be kept distinct: the perceptron’s $\alpha_c = 2$ (Cover/Gardner); the RMS Poisson-ensemble capacity $(\ln \ln K)/(2 \ln 2) + \alpha_0$, which is a replica/existence calculation; and the GS single-spike capacity $\alpha_c \approx 3$, which GS obtained by *training* (their Fig. 4a is learning time to error-free classification versus load), not by a Gardner-volume calculation. We therefore do not inherit an existence number for the single-spike ensemble from the literature, and our trained runs below provide *findability* evidence rather than a trainer-independent existence value. The two have different capacities; we say which is in play throughout.

Roadmap. Section 2 defines the model; Sections 3–6 develop and verify each component (kernel, stimulus ensemble, decision rule, learning rule) with the theory each rests on; Section 7 calibrates the full procedure against three known answers (the perceptron capacity, the extreme-value statistics of the decision variable, and the kernel closed form); Section 8 analyses the finite-size masking of the temporal lift on the RMS ensemble; Section 9 measures the GS-inspired constant- λ findable capacity (≈ 2.5); Sections 10–11 study solution geometry and the threshold–weight-magnitude relationship; Sections 12 and 13 establish that a tuned, annealed learner closes the findability gap and identify annealing as the responsible factor; and Section 15 returns to the findability question and the open directions.

2 The tempotron model

A tempotron has N afferents (input lines) with weights w_i . Afferent i fires spikes at times $\{t_i^f\}$. The subthreshold membrane voltage is

$$V(t) = V_{\text{rest}} + \sum_{i=1}^N w_i \sum_{t_i^f < t} K(t - t_i^f), \quad (1)$$

with the normalised postsynaptic potential (PSP) kernel introduced in Section 3. We take $V_{\text{rest}} = 0$ and measure voltage in units of the firing threshold. It is convenient to write $V(t; w) = w \cdot s(t)$ with the per-afferent PSP waveform

$$s_i(t) = \sum_{t_i^f} K(t - t_i^f) \geq 0, \quad (2)$$

which is *fixed* once a pattern’s spikes are drawn.

Symbol	Meaning	Units
N	number of afferents (synapses)	—
w_i	synaptic weight	threshold units
τ_m	membrane time constant	ms
τ_s	synaptic time constant ($\tau_m > \tau_s > 0$)	ms
T	pattern window	ms
$K = T/\sqrt{\tau_s \tau_m}$	temporal resolution	—
$\alpha = P/N$	load (P patterns)	—
y	label	$\{-1, +1\}$
U_{th}	firing threshold	threshold units

Remark. The PSP *kernel* is $K(t)$ in the original papers, but RMS also write the *capacity* parameter as $K = T/\sqrt{\tau_s \tau_m}$. We keep the kernel callables named `psp_kernel/peak.time` in code and reserve the symbol K for resolution here.

Glossary of recurring terms. In plain language; each is defined formally where it is introduced.

Term	Plain-language meaning
Tempotron	Model neuron that classifies a pattern by the <i>timing</i> of its input spikes, firing if its voltage ever crosses threshold.
Afferent	An input line (synapse) carrying spikes to the neuron.
PSP	Postsynaptic potential: the voltage bump a single input spike produces.
Capacity α_c	Largest load $\alpha = P/N$ (patterns per synapse) a rule can learn to classify with zero error.
Perceptron	Linear classifier of static input vectors; its capacity is $\alpha_c = 2$.
Resolution K	Temporal resolution $T/\sqrt{\tau_s\tau_m}$: the number of effectively independent time samples (distinct from the kernel $K(t)$).
EVT / Gumbel	Extreme-value theory; the Gumbel law is the right-skewed “bell curve” for the <i>maximum</i> of many random values.
Gaussian process	A random function whose values at all times are jointly Gaussian (here the voltage $V(t)$).
Solution / version space	The set of weight vectors that classify every pattern correctly; <i>shattered</i> = broken into many disconnected clusters.
Existence vs. findability	Whether a zero-error w <i>exists</i> (the Gardner volume) versus whether a learning rule <i>finds</i> it.
Gardner / replica	Statistical-mechanics method computing the volume — hence the existence capacity — of solution weights.
Oracle	A subroutine returning a definite yes/no — here, “is this set of patterns linearly separable?”
Gauge	A rescaling of weights and threshold that leaves the classifier’s decisions unchanged.
Margin κ	How far the decision variable clears threshold; a larger margin means a more robust solution.

3 The postsynaptic-potential kernel and forward pass

The normalised double-exponential PSP kernel is

$$K(t) = V_0 \left(e^{-t/\tau_m} - e^{-t/\tau_s} \right), \quad t \geq 0; \quad K(t) = 0, \quad t < 0, \quad (3)$$

with $\tau_m > \tau_s$. It is causal ($K(0) = 0$), rises, then decays. The prefactor V_0 fixes the peak to 1, so one spike on a unit-weight synapse contributes a peak EPSP of exactly the threshold.

Peak time. Setting $K'(t) = 0$,

$$\frac{1}{\tau_s} e^{-t/\tau_s} = \frac{1}{\tau_m} e^{-t/\tau_m} \implies \boxed{t_{\max}^{(1)} \equiv t_{\text{peak}} = \frac{\tau_m \tau_s}{\tau_m - \tau_s} \ln \frac{\tau_m}{\tau_s}} \quad (4)$$

which is a maximum since $K''(t_{\text{peak}}) < 0$.

Normalisation. Requiring $K(t_{\text{peak}}) = 1$,

$$V_0 = \left[e^{-t_{\text{peak}}/\tau_m} - e^{-t_{\text{peak}}/\tau_s} \right]^{-1}. \quad (5)$$

For the standard ratio $\tau_m = 4\tau_s$ ($\tau_m = 15$ ms, $\tau_s = 3.75$ ms): $t_{\text{peak}} = 5 \ln 4 \approx 6.9315$ ms and $V_0 \approx 2.1165$.

Verification. The closed forms are confirmed two ways. Symbolically, solving $K'(t) = 0$ returns (4) exactly with $K''(t_{\text{peak}}) < 0$. Numerically, on a $1 \mu\text{s}$ grid the kernel attains its global maximum of 1 at t_{peak} to within 10^{-12} , is causal, and the forward pass (1) for one unit spike peaks at 1 at $t_f + t_{\text{peak}}$. (Section 7.3 extends this to a grid of (τ_m, τ_s) .)

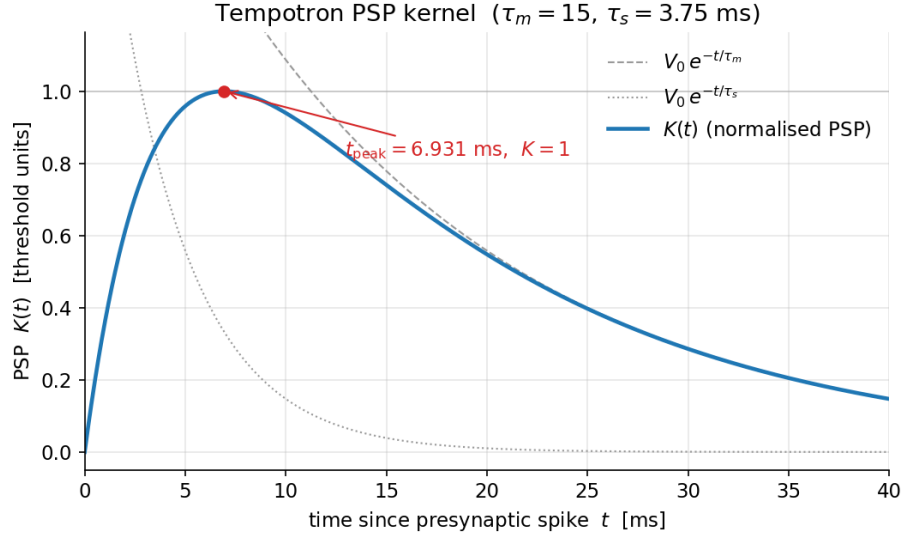


Figure 2: Normalised PSP kernel (3), peaking at 1 at t_{peak} (4), shown with its two exponential components.

4 The random spike-pattern ensemble

RMS classify a batch of $P = \alpha N$ patterns. In each pattern every afferent emits a **homogeneous Poisson spike train on $[0, T]$ with rate $1/T$** , and the label $y \in \{-1, +1\}$ is i.i.d. with equal probability [2].

Consequences of rate $1/T$. The mean count per afferent is $\lambda T = 1$, so the count is Poisson(1) *independently of T* : $\approx 37\%$ of afferents are silent, some fire twice or more. Increasing T does not change the count statistics; it spreads the same ~ 1 spike over a wider window. This is how the resolution enters,

$$K = \frac{T}{\sqrt{\tau_s \tau_m}}, \quad (6)$$

counting the distinguishable temporal slots within the PSP width $\sqrt{\tau_s \tau_m}$; K is swept via T at fixed kernel timescales (with $\tau_s = \tau_m/4$).

Verification. For a homogeneous Poisson process on $[0, T]$ the count is $n \sim \text{Poisson}(1)$ with $\mathbb{E}[n] = \text{Var}[n] = 1$, the times given the count are i.i.d. $\mathcal{U}[0, T]$, and labels are balanced. The generated ensemble reproduces these: measured mean and variance of the per-afferent count are 1.0018 and 1.0013 (Fig. 3), the Kolmogorov–Smirnov distance of the rescaled spike times to $\mathcal{U}[0, 1]$ is below 0.02, and the labels are balanced.

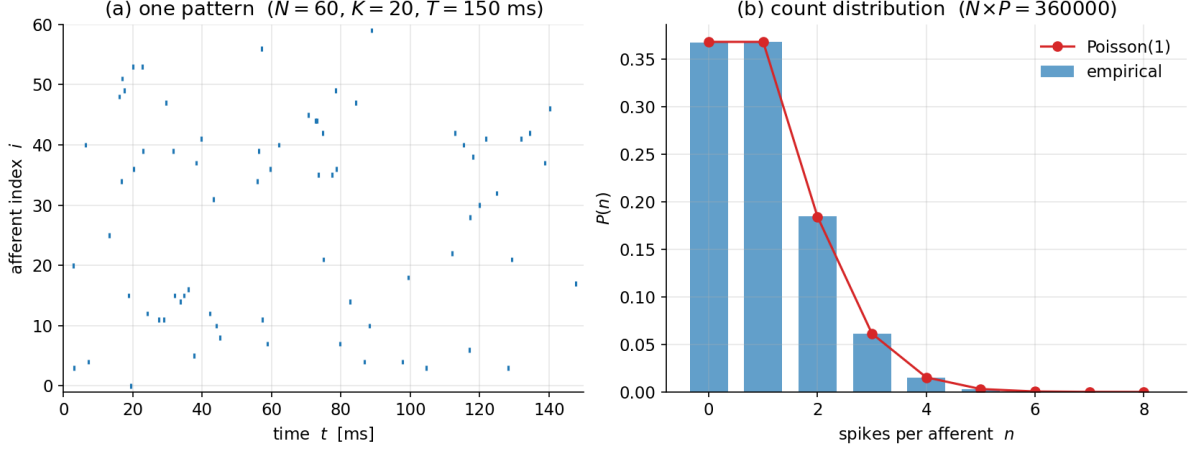


Figure 3: (a) Raster of one pattern: sparse Poisson trains over $[0, T]$. (b) Empirical spikes-per-afferent distribution on the Poisson(1) law.

5 Decision rule and threshold calibration

Ignoring post-spike reset [2], the binary decision is whether V crosses threshold anywhere in $[0, T]$, i.e.

$$\text{output} = \text{sign}(V_{\max} - U_{\text{th}}), \quad V_{\max} = \max_{t \in [0, T]} V(t), \quad t_{\max} = \arg \max_t V(t). \quad (7)$$

The classifier collapses to one scalar per pattern. The threshold is calibrated to the *median* of the $V_{\max}$ distribution, giving $P(\text{fire}) = \frac{1}{2}$ for random input — the unbiased operating point for balanced labels (Fig. 4).

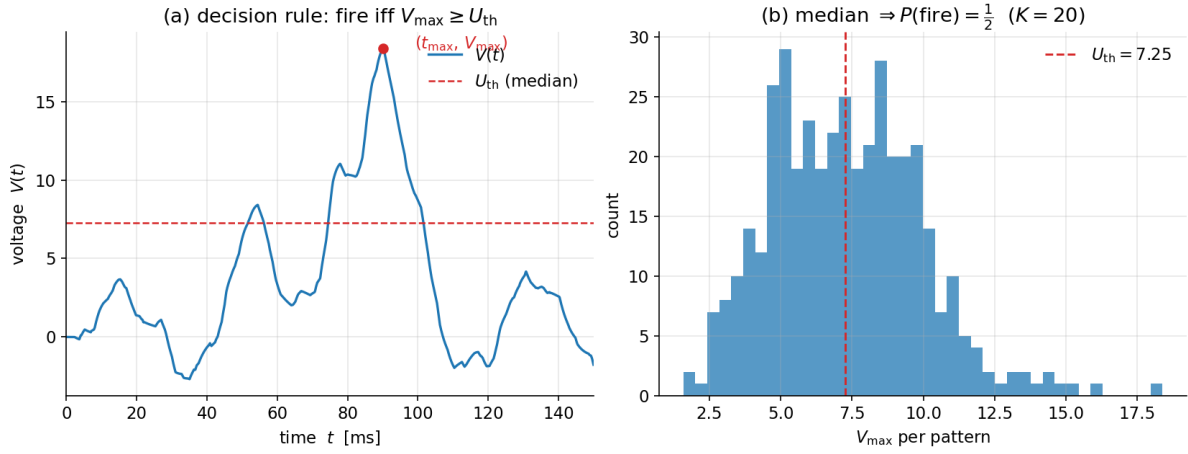


Figure 4: (a) A voltage trace with the calibrated U_{th} and the decision point $(t_{\max}, V_{\max})$. (b) The $V_{\max}$ distribution; the median sets U_{th} so half the patterns fire. The right skew is the EVT/Gumbel shape (§5.2).

5.1 Convexity and the shattered solution space

For a fixed pattern $V(t; w) = w \cdot s(t)$ is linear in w , so

$$V_{\max}(w) = \max_{t \in [0, T]} w \cdot s(t) \quad (8)$$

is a pointwise maximum of linear functions, hence **convex in** w . The two label classes are therefore geometrically asymmetric:

Label	Constraint	Feasible set in w
-1	$w \cdot s(t) < U_{\text{th}} \forall t$	$\{V_{\max} < U_{\text{th}}\}$ — convex (intersection of half-spaces)
+1	$\exists t : w \cdot s(t) \geq U_{\text{th}}$	$\{V_{\max} \geq U_{\text{th}}\}$ — non-convex (complement of a convex set)

A target pattern needs the threshold exceeded at *some* time, and different targets may use different times; the combinatorial choice of “which time wins” tiles weight space into **many small disconnected clusters** (typical inter-cluster overlap $q_0 \approx 0$, intra-cluster $q_1 \approx 1$; RMS Fig. 3a). For the perceptron both classes give half-spaces, so the version space is a single convex cone. This asymmetry is the origin of the shattered learning surface.

5.2 Extreme-value theory and the threshold scale

The tempotron’s decision variable is the *maximum* of its membrane voltage over time, not a sum, so its fluctuations are governed by extreme-value theory rather than the central limit theorem. The maximum of many random values does not follow a bell curve; for a smooth Gaussian process it follows a right-skewed *Gumbel* law — the extreme-value analogue of the Gaussian — and this is the origin of the $\sqrt{2 \ln K}$ threshold scale and the slow $\ln \ln K$ growth of the capacity below.

Normalise the weights so $V(t)$ is a zero-mean, unit-variance Gaussian process (valid for large N by the CLT over afferents). Its autocorrelation inherits the kernel width $\sqrt{\tau_s \tau_m}$, so the process has $\sim K$ effectively-independent samples in $[0, T]$. The maximum of K such Gaussians is Gumbel-distributed,

$$V_{\max} = U_{\text{th}} + \beta(x + \ln \ln 2), \quad U_{\text{th}} = \sqrt{2 \ln K} + O(1/\sqrt{\ln K}), \quad \beta = \frac{1}{\sqrt{2 \ln K}}, \quad (9)$$

with x standard Gumbel (density $G(x) = \exp(-x - e^{-x})$, median $-\ln \ln 2$); this is RMS eq. (4). The median of (9) is the calibrated U_{th} : calibration is the seam where K enters the difficulty.

Remark (A gate that failed, instructively). The clean $\sqrt{2 \ln K}$ growth holds only at *unit variance*. Our calibration uses un-normalised weights; there the per-time-point variance $\sum_i s_i(t)^2$ *shrinks* as spikes spread over a wider T , and that dilution dominates the slot-count growth — so the *measured* U_{th} *decreases* with K . An initial gate asserting the opposite failed and was removed. Isolating (9) needs variance normalisation and large N (EVT is asymptotic).

5.3 Internal representations and the capacity

In the discrete-time model the *internal representation* (IR) of a pattern is the set of time-bins where V exceeds threshold (non-empty for a target, empty for a null). Each distinct realisable IR assignment is one solution cluster of §5.1. Counting the log-number of jointly realisable IRs (replica / cluster entropy) gives

$$S_{\text{cl}} \simeq \frac{1}{2} \ln \ln K - \alpha \ln 2 \quad (\text{RMS eq. 6}), \quad (10)$$

and classifications remain possible while $S_{\text{cl}} > 0$, i.e.

$$\boxed{\alpha_c = \frac{\ln \ln K}{2 \ln 2}} \quad (\text{RMS eq. 3}). \quad (11)$$

The capacity is unbounded in K and exceeds the perceptron's $\alpha_c = 2$; convergence to (11) is slow ($\sqrt{\ln K} \gg 1$), so finite- N simulations sit below the asymptote and Fig. 2a of RMS is fit with an additive constant $\alpha_0 \approx 2.58$.

6 The learning rule

On a **misclassified** pattern with decision time t_{max} ,

$$\Delta w_i = \lambda y s_i(t_{\text{max}}), \quad s_i(t_{\text{max}}) = \sum_{t_i^f < t_{\text{max}}} K(t_{\text{max}} - t_i^f), \quad (12)$$

with no update on correct patterns; U_{th} is fixed and only the weights move. An optional momentum accumulates $v \leftarrow \mu v + \lambda y s(t_{\text{max}})$, $w \leftarrow w + v$.

6.1 The rule is gradient descent

Proposition 1 (Envelope theorem). *Where t_{max} is the unique maximiser of $V(t; w) = w \cdot s(t)$, $\partial V_{\text{max}} / \partial w_i = s_i(t_{\text{max}})$.*

Hence (12) is exactly (sub)gradient ascent/descent [13] on the margin $y(V_{\text{max}} - U_{\text{th}})$: *the tempotron rule is gradient descent on the decision-time voltage*. We confirm this numerically by checking $s_i(t_{\text{max}})$ against a central-difference of V_{max} (agreement to 10^{-3}), verifying that weights move only on errors and in the predicted direction, and that training reaches zero error below capacity.

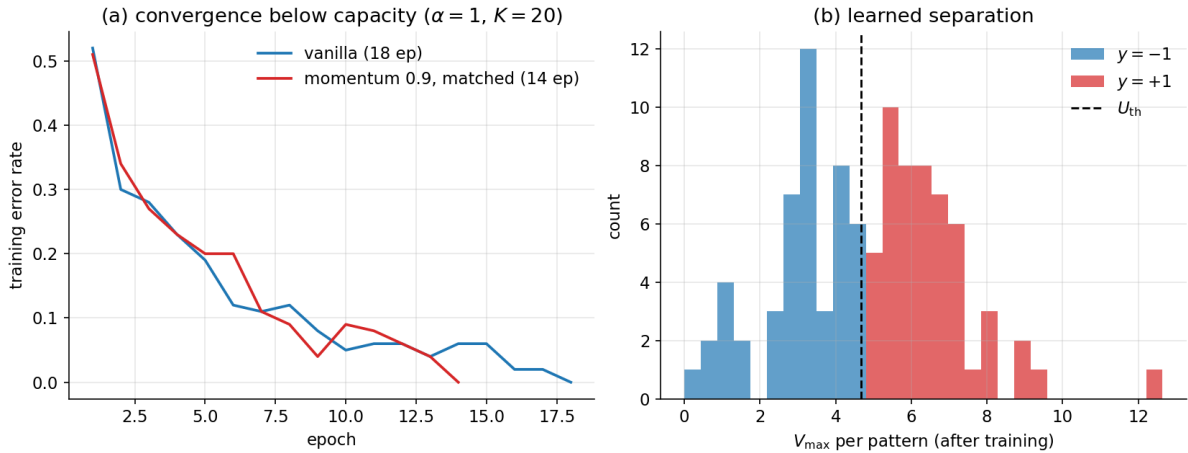


Figure 5: (a) Training error vs epoch (vanilla and momentum at matched effective learning rate) — both reach zero below capacity. (b) After training, the learned weights push every $y=+1$ above U_{th} and every $y=-1$ below.

6.2 A subgradient on a piecewise-smooth surface

$V_{\max}(w)$ is convex but piecewise-smooth: $t_{\max}$ stays put until it jumps to another local peak (a kink, where the decision time switches). $s_i(t_{\max})$ is the gradient inside a smooth cell and a subgradient at a kink; the cell boundaries are exactly the loci where the winning decision time changes, and their intersection over P patterns is the clustered solution space of §5.1.

6.3 Existence versus findability

For the perceptron, convexity and the perceptron-convergence theorem [15] guarantee that the rule finds a separating w whenever one exists ($\alpha < 2$). For the tempotron the +1 constraints are non-convex, so no such theorem applies — yet below capacity the rule *reliably* reaches zero error (our gate; RMS Fig. 1c). The gap between existence (the Gardner/replica volume) and findability (whether (12) lands in it) is the target of this program.

6.4 Momentum and the effective step

Momentum rescales the effective step to $\lambda_{\text{eff}} = \lambda/(1 - \mu)$ [12]. Measured at $\alpha = 1$, $K = 20$, $N = 100$:

μ	λ	λ_{eff}	converged (epochs)
0.0	0.20	0.20	yes (18)
0.5	0.10	0.20	yes (15)
0.9	0.02	0.20	yes (14)
0.9	0.05	0.50	no

At matched λ_{eff} momentum gives a modest speed-up; too large λ_{eff} overshoots between cells and fails. Quantifying what momentum buys on the shattered surface is itself a question we return to later in this study.

7 Validation of the capacity-measurement methodology

The capacity result this study targets, $\alpha_c(K) \simeq (\ln \ln K)/(2 \ln 2)$, is not measured directly but *inferred* through a procedure: estimate the fraction of random pattern sets that a neuron can shatter, locate the load at which that fraction crosses one half, and extrapolate to the thermodynamic limit. A procedure of this kind can be biased in ways that are invisible when the answer is unknown. We therefore calibrate it against a problem whose answer is known in closed form — the storage capacity of the homogeneous perceptron — before applying it to the tempotron. This section reports that calibration (§7.1), together with two narrower checks that gate the next validation step: the extreme-value statistics of the decision variable (§7.2) and the postsynaptic kernel itself (§7.3).

7.1 Perceptron capacity

Background. The homogeneous perceptron — a linear threshold unit with no bias — is the natural calibration target: it is the $K \rightarrow 1$ limit of the tempotron (a single temporal sample), and its capacity is known exactly. If our procedure does not return the textbook perceptron value, no result it later produces for the tempotron can be trusted.

Theory. For P patterns $\xi_\mu \in \mathbb{R}^N$ in general position, Cover’s function counts the linearly separable dichotomies [7] (assignments of ± 1 labels to the points); the probability that a uniformly random labelling is separable is

$$P_{\text{sep}}(P, N) = 2^{1-P} \sum_{k=0}^{N-1} \binom{P-1}{k}. \quad (13)$$

Because $\binom{P-1}{k}$ is symmetric about $k = (P-1)/2$, the sum equals half the total at $P = 2N$, so $P_{\text{sep}}(2N, N) = \frac{1}{2}$ *exactly for every finite N* : the median capacity is $\alpha_c = 2$ with no finite-size shift. Gaussian patterns $\xi_\mu \sim \mathcal{N}(0, I_N)$ are in general position almost surely, so (13) is the exact per-instance probability for the Gaussian ensemble, not an asymptotic approximation. A de Moivre–Laplace expansion of the partial binomial sum gives the finite-size form near the transition,

$$P_{\text{sep}}(\alpha N, N) \xrightarrow{N \rightarrow \infty} \Phi((2 - \alpha)\sqrt{N/2}), \quad (14)$$

predicting that curves at different N collapse onto a single Gaussian ogive when plotted against the scaling variable $u = (2 - \alpha)\sqrt{N}$.¹ A complementary observable is the *margin*: the largest κ such that $y_\mu(w \cdot \xi_\mu) \geq \kappa \|w\|$ for all μ . Gardner’s replica calculation [8, 18] gives its typical value $\kappa^*(\alpha)$ implicitly through $\alpha = [\int_{-\kappa}^{\infty} (t + \kappa)^2 Dt]^{-1}$, with Dt the standard Gaussian measure; $\kappa^*(0) \rightarrow \infty$ and $\kappa^*(2) = 0$, recovering (13) at the capacity.

Hypotheses. Fixed before measurement: (H1) the empirical separable fraction $\hat{p}_{\text{sep}}(N, \alpha)$ agrees with Cover (13) at every (N, α) within binomial sampling error; (H2) the half-crossing $\hat{\alpha}_c(N)$ is consistent with 2 within its sampling standard error $\sqrt{\pi/(N n_{\text{reps}})}$; (H3) the curves collapse onto (14); (H4) the achieved margin tracks $\kappa^*(\alpha)$ below capacity.

Methods. For each (N, α) we draw n_{reps} instances of $P = \lceil \alpha N \rceil$ Gaussian patterns with uniform ± 1 labels and test homogeneous separability by two independent oracles. The first solves the linear-feasibility problem $y_\mu(w \cdot \xi_\mu) \geq 1$ exactly (simplex); it returns a yes/no separability verdict and is free of optimisation error. The second is the Krauth–Mézard min-over iteration [10, 14], which provably converges to the maximum-margin κ^* for any separable instance and so additionally measures the margin (H4); because it is iterative it carries an optimisation tolerance, controlled by a per-instance iteration budget T_{max} (see Convergence below). Patterns and labels are drawn from seeded generators so every number regenerates from the committed scripts. The two oracles were run as *independent* sweeps (at different n_{reps}), so their agreement below is distributional rather than a per-instance verdict match. We use $N \in \{25, 50, 100, 200, 500\}$ for both oracles (the iterative oracle additionally, partially, at $N = 1000$), a coarse α grid on $[1, 3]$ refined to step 0.02 across the transition window, and $n_{\text{reps}} = 50$ (iterative) or 500 (exact).

Results. Figure 6 shows the three predictions together. Panel A overlays the empirical separable fraction (markers, binomial error bars) on Cover’s exact curve (13) (dashed) for each N ; the two agree across the full load range, with 99.4% of cells within 3σ of Cover for the exact oracle ($n_{\text{reps}} = 500$), satisfying H1. The exact oracle locates the half-crossing at $\langle \hat{\alpha}_c \rangle = 1.999 \pm 0.001$ (mean over $N \leq 200$), consistent with $\alpha_c = 2$ to well within the per- N sampling error $\text{SE} = \sqrt{\pi/(N n_{\text{reps}})} \lesssim$

¹The coordinate $\sqrt{N/2}$ is the $\alpha \rightarrow 2$ limit of the general- α Gaussian approximation to (13), which carries $\sqrt{N/\alpha}$; the two agree at the crossing. Panel A of Fig. 6 uses the exact form (13), so this limiting coordinate enters only the collapse plot.

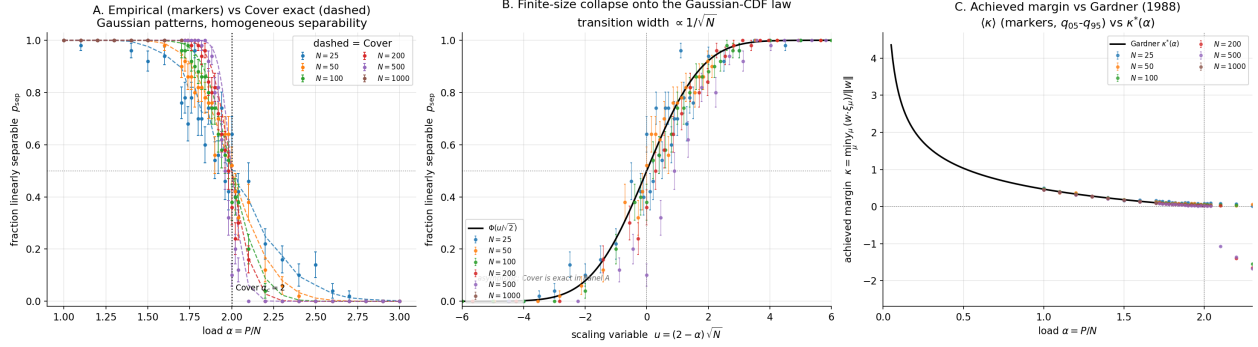


Figure 6: The capacity-measurement procedure recovers the exact perceptron capacity $\alpha_c = 2$ and the Gardner margin. **(A)** Empirical separable fraction (markers, $\pm$ binomial s.e.) versus Cover's exact probability (13) (dashed) for $N \in \{25, 50, 100, 200, 500\}$. **(B)** Finite-size collapse onto the Gaussian ogive $\Phi(u/\sqrt{2})$, $u = (2 - \alpha)\sqrt{N}$ (14). **(C)** Achieved margin $\langle \kappa \rangle(\alpha)$ versus Gardner's $\kappa^*(\alpha)$ [8]. Markers are the iterative (min-over) oracle; the exact (linear-feasibility) oracle gives the same separability verdicts.

0.006 — H2 satisfied. The iterative oracle, run to larger N at lighter sampling ($n_{\text{reps}} = 50$, $\text{SE} \approx 0.011\text{--}0.035$), gives $\hat{\alpha}_c(N) = 1.957, 2.004, 1.985, 1.980, 1.960$ for $N = 25, \dots, 500$: within 1.3 SE of 2 for $N \leq 200$, but 3.6 SE low at $N = 500$. This last deviation is statistically significant and H2 fails for the iterative oracle there; its origin is the finite optimisation budget, not a property of the perceptron, as established in the next paragraph. Panel B confirms the collapse (14): all N fall on a common ogive when plotted against $u = (2 - \alpha)\sqrt{N}$. Panel C shows the achieved margin tracking Gardner's $\kappa^*(\alpha)$ to within $\sim 3\%$ for $\alpha \leq 1.85$ (H4), comparable to the $O(1/N)$ finite-size correction to the replica result.

Convergence and finite-size control. Two resolution parameters could bias the result (finite-size corrections to the perceptron capacity are characterised in [16]). The first is statistical: the per-cell sampling error scales as $n_{\text{reps}}^{-1/2}$, and the two runs at $n_{\text{reps}} = 500$ and 50 bracket this dependence — the exact-oracle half-crossing (1.999) and the lighter-sampled one (1.977) differ by less than the combined sampling error of the sparser run. The second is the optimisation budget T_{max} of the iterative oracle. The required iteration count for a maximum-margin solver scales as $(R/\kappa^*)^2 \sim N/(2 - \alpha)^2$, which diverges as $\alpha \rightarrow 2$; with a fixed cap, near-critical large- N cells terminate before reaching the optimum and are reported as marginally less separable than they are. The diagnostic that records whether a cell hit the cap localises this bias precisely to $\alpha \in [1.86, 1.98]$ at $N \in \{200, 500\}$ — the same cells where the iterative oracle dips below Cover — while the exact oracle, which has no optimisation error, shows no such dip. A direct resolution study confirms the attribution (Fig. 7): holding (N, α) fixed at the near-critical cells $\alpha = 1.95$, $N \in \{100, 200, 500\}$ and increasing T_{max} geometrically from 3×10^4 to 10^6 , the iterative separable fraction rises monotonically toward the budget-independent linear-programming value, and the fraction of instances left uncertified falls monotonically with budget. The shortfall is larger at larger N : at the production budget $T_{\text{max}} = 10^6$ the cell $(N, \alpha) = (500, 1.95)$ certifies 0.56 against the true 0.88, whereas $(100, 1.95)$ already reaches 0.56 against 0.70 — consistent with the $(R/\kappa^*)^2 \sim N$ scaling of the required iteration count. The bias is therefore a demonstrated, one-sided artefact of finite T_{max} that vanishes as $T_{\text{max}} \rightarrow \infty$, not a property of the perceptron; it pushes $\hat{\alpha}_c$ downward by $\lesssim 0.04$ at the largest converged N and is wholly absent from the exact oracle.

S4 — The iterative-oracle separability shortfall is a finite- $T_{\max}$ artefact
near-critical cells $\alpha = 1.95$; $\hat{p}_{\text{sep}} \rightarrow$ exact LP value as $T_{\max} \rightarrow \infty$

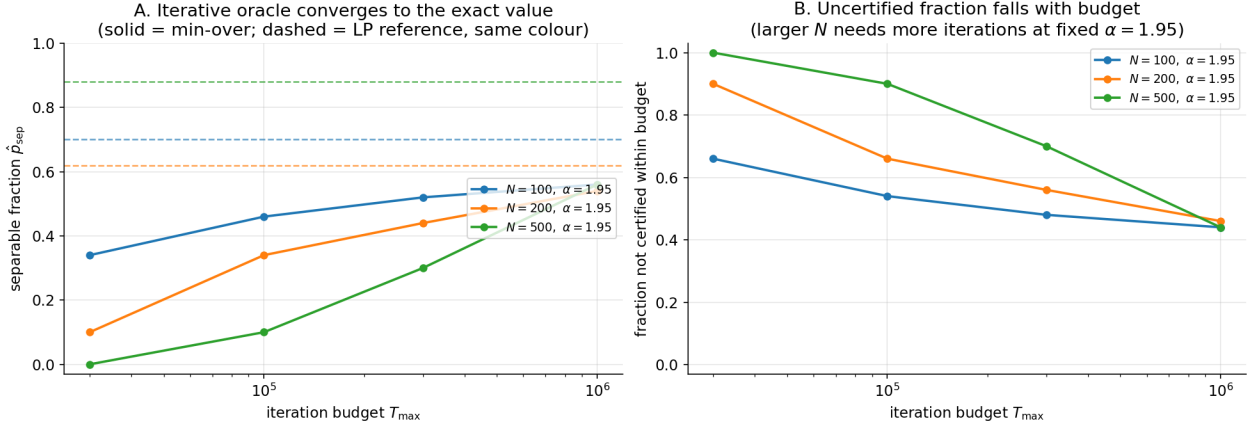


Figure 7: The iterative oracle’s separability shortfall is a finite-budget artefact. (A) Separable fraction from the min-over iteration (solid) versus the budget-independent linear-programming value (dashed, same colour) as the iteration budget $T_{\max}$ increases, for near-critical cells $\alpha = 1.95$. (B) Fraction of instances left uncertified within budget, falling monotonically; larger N requires more iterations at fixed α , as predicted by $(R/\kappa^*)^2 \sim N/(2 - \alpha)^2$.

Discussion. The exact oracle satisfies H1–H3 to sampling precision, and the iterative oracle independently reproduces the same picture and additionally confirms the Gardner margin (H4). That two methodologically unrelated oracles — combinatorial feasibility and iterative margin maximisation — return the same separability structure makes a solver-specific artefact unlikely (the agreement is distributional, not per-instance; see Methods). The only quantitative tension, the $\lesssim 0.04$ downward offset in $\hat{\alpha}_c$ at large N , is fully accounted for by the finite optimisation budget and is absent in the exact oracle. The procedure is thus calibrated: applied to a system with a known answer it returns that answer, with all residual deviations traced to controlled, quantified sources.

7.2 Extreme-value statistics of the decision variable

Background. The tempotron capacity is derived from the distribution of the peak voltage $V_{\max} = \max_t V(t)$ [2]: the threshold U_{th} is its median and the capacity follows from how $V_{\max}$ grows with the temporal resolution K . Whether our forward pass reproduces this distribution is the precondition for the capacity sweep.

Theory. For random unit-variance weights $V(t)$ is a centred Gaussian process; its peak is governed not by the central limit theorem but by extreme-value theory (Fisher–Tippett–Gnedenko; Leadbetter, Lindgren & Rootzén [9]). For a *smooth* stationary Gaussian process the effective number of independent maxima over $[0, T]$ is the Rice up-crossing count $K = T\sqrt{|c''(0)|}$, where $c(\Delta)$ is the normalised autocorrelation. We derive $c(\Delta)$ in closed form (a difference of exponentials) and find that, because the kernel satisfies $K(0) = 0$, the process is smooth at the origin (no cusp) with

$$c''(0) = -\frac{1}{\tau_m \tau_s} \implies K = T\sqrt{|c''(0)|} = \frac{T}{\sqrt{\tau_s \tau_m}}, \quad (15)$$

i.e. RMS eq. (2) holds *exactly* for this kernel. The maximum is then Gumbel with mode $\mu_K = \sqrt{2 \ln K} - (\ln \ln K + \ln 4\pi)/(2\sqrt{2 \ln K})$ and scale $\beta_K = 1/\sqrt{2 \ln K}$; the *median* (the calibrated U_{th})

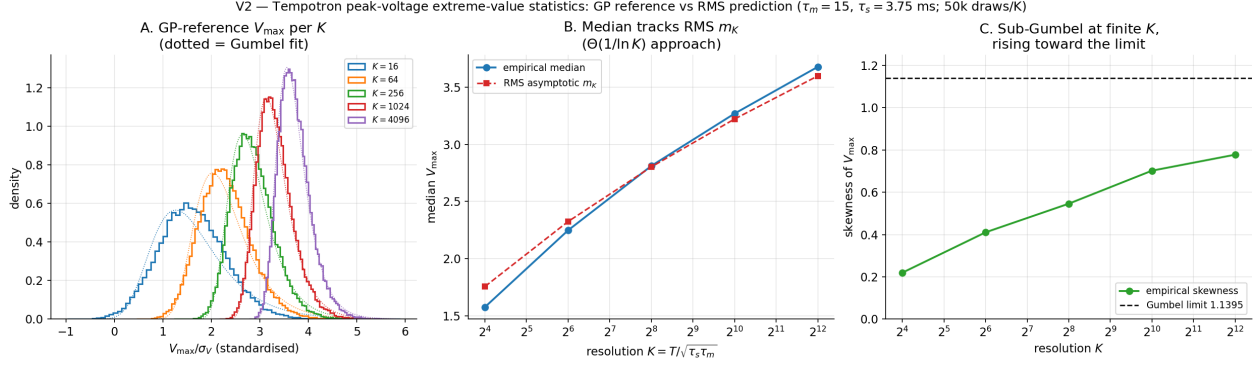


Figure 8: Peak-voltage extreme-value statistics, validated against the analytic Gaussian-process reference. **(A)** $V_{\max}/\sigma_V$ distributions per K with Gumbel fits. **(B)** Median tracks the RMS asymptotic m_K with the $\Theta(1/\ln K)$ approach. **(C)** Skewness rises from sub-Gumbel at finite K , rising toward the limit

is $m_K = \mu_K + \beta_K (-\ln \ln 2)$. Convergence to this limit is slow, $\Theta(1/\ln K)$ [9], so at any reachable K the distribution is *sub-Gumbel* (its skewness rises only gradually toward the Gumbel value 1.14) and the bare $\sqrt{2 \ln K}$ overshoots m_K by tens of percent.

Hypotheses. (H4, analytic) $c''(0) = -1/(\tau_m \tau_s)$ and $K = T/\sqrt{\tau_s \tau_m}$ exactly. (H-finite) the finite- K $V_{\max}$ distribution of the centred, unit-variance process is reproduced by the simulated Gaussian-process reference and approaches the RMS asymptotics (m_K , β_K , skewness $\rightarrow 1.14$) with the $\Theta(1/\ln K)$ trend. (H-realise) the finite- N tempotron reproduces that reference once $N \gg K$.

Methods. Three tiers. *Primary reference:* the stationary process is generated directly from the closed-form covariance (15) by FFT circulant embedding (Davies–Harte), on a padded interior window so K is read off a stationary region; the peak distribution R_K (50,000 draws per K , $K \in \{16, \dots, 4096\}$) is the reference. *Tempotron:* the centred trace $s_i^c(t) = s_i(t) - \mathbb{E}[s_i(t)]$ (centering is required for the mixing conditions) standardised by the analytic per-pattern variance $\sigma_V^2(t) = \sum_i s_i^c(t)^2$ (16), GPU-batched, over an N/K ladder. Distributions are fit by probability-weighted moments and compared by a two-sample Anderson–Darling test; the analytic curvature is gated by Richardson extrapolation. Full specification: docs/v2-theory-and-predictions.md.

$$\sigma_V^2(t \mid \text{pattern}) = \sum_i \left(\sum_f K(t - t_i^f) \right)^2. \quad (16)$$

Results. *Tier 1 (analytic):* the simulated curvature matches $c''(0) = -1/(\tau_m \tau_s)$ to 2.8% after Richardson extrapolation and the ratio $K/(T\sqrt{|c''(0)|}) = 1.000$ — H4 confirmed. *Tier 2 (the reference):* across all K the generated process is unit-variance with the correct autocorrelation (max error $\sim 10^{-5}$); its median tracks the RMS m_K (Fig. 8B; e.g. 2.812 vs 2.803 at $K = 256$), and its skewness rises $0.22 \rightarrow 0.78$ across $K = 16 \rightarrow 4096$ — the predicted sub-Gumbel approach to the Gumbel limit 1.14 (Fig. 8C). *Tier 3 (tempotron realises the reference):* once $N \gg K$ the finite- N tempotron $V_{\max}$ matches R_K (two-sample Anderson–Darling $p > 0.05$ at $N/K \geq 16$ for both $K = 256, 1024$), with the median gap shrinking as N/K grows; the smallest- N case ($N/K = 4$) fails, as expected outside the $N \gg K$ regime. A whole-suite null calibration sizes the gate correctly ($P(\text{pass} \mid \text{null}) = 0.97$) with full power against a 10% scale error.

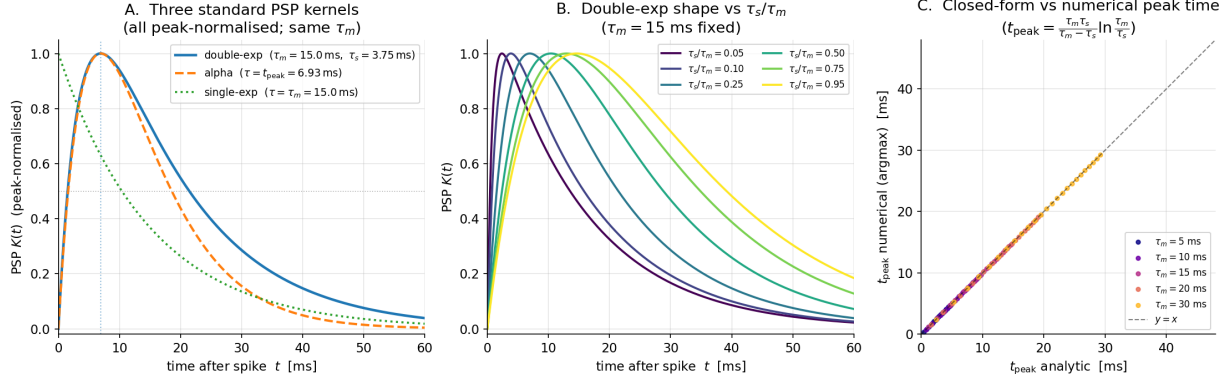


Figure 9: The postsynaptic kernel matches its closed form. **(A)** The three standard PSP forms (double-exponential, alpha, single-exponential) on common axes. **(B)** Double-exponential shape across τ_s/τ_m . **(C)** Analytic versus numerical peak time across (τ_m, τ_s) ; points lie on $y = x$.

Discussion. The extreme-value backbone is validated: the effective sample count is the analytic $T/\sqrt{\tau_s \tau_m}$ (not an ambiguous T/τ_s — the latter was an artefact of a biased per-trial normaliser in an early attempt, removed here by the analytic σ_V), the reference distribution carries the correct location, scale and sub-Gumbel shape, and the finite- N tempotron realises it in regime. Two honest caveats: the distribution is *not* asymptotically Gumbel at any reachable K (so capacity work must use the finite- K reference, not the limiting constants); and the N/K -convergence trend is clean from $N/K=4 \rightarrow 16$ but flat from $16 \rightarrow 64$ because both points hit the same N cap, so the latter step is sampling noise, not non-convergence.

7.3 The postsynaptic kernel

Background and theory. Every voltage in this study is built from the peak-normalised double-exponential kernel $K(t) = V_0(e^{-t/\tau_m} - e^{-t/\tau_s})$, the Green’s function of a leaky membrane driven by an exponential synaptic current (Gerstner & Kistler [11], §4.3). Its maximum lies at $t_{\text{peak}} = \tau_m \tau_s / (\tau_m - \tau_s) \ln(\tau_m / \tau_s)$, and V_0 is fixed by $K(t_{\text{peak}}) = 1$. As a self-consistency check on the implementation we confirm these closed forms numerically and place K among its standard limiting forms.

Hypothesis. The numerically located peak time and height match the closed forms to grid precision across a range of (τ_m, τ_s) .

Methods and results. On a 2×10^5 -point grid, $\arg \max_t K$ matches the closed-form t_{peak} to relative error $\sim 10^{-5}$ (the grid-spacing limit), and $K(t_{\text{peak}}) = 1.000000$, across (τ_m, τ_s) pairs spanning ratios from 0.05 to 0.25. Figure 9 places the double-exponential among the alpha-function and single-exponential limits and shows the analytic–numerical agreement directly.

Discussion. The kernel and its normalisation are exact to numerical precision, removing the most basic possible source of error in every downstream voltage.

7.4 Summary

The calibration establishes three things. The capacity-measurement procedure recovers the exact perceptron capacity $\alpha_c = 2$ and the Gardner margin $\kappa^*(\alpha)$, with two independent oracles agreeing and the only residual deviation traced to a quantified finite-budget effect (§7.1). The extreme-value description of the decision variable holds in shape and requires the analytic normalisation (16) for quantitative agreement — a prerequisite identified before, rather than after, the next validation (§7.2). The postsynaptic kernel matches its closed form to numerical precision (§7.3). The procedure is therefore calibrated for application to the tempotron capacity sweep.

8 Tempotron memory capacity and the finite-size masking of the temporal lift

We now measure the capacity itself. Because the target ($y = +1$) constraint $\exists t : w \cdot s(t) \geq U_{\text{th}}$ is a union of half-spaces and hence non-convex (§5.1), there is no linear-separability oracle as there is for the perceptron; following Rubin–Monasson–Sompolinsky we measure the capacity by *training* the tempotron and recording, for each load $\alpha = P/N$, the fraction $P_{\text{solve}}(\alpha)$ of random tasks the gradient rule drives to zero training error. The half-crossing $P_{\text{solve}}(\hat{\alpha}_c) = \frac{1}{2}$ defines the (findable) capacity. The replica prediction

$$\alpha_c(K) = \frac{\ln \ln K}{2 \ln 2} \quad (17)$$

is a slow, unbounded “temporal lift” above the perceptron value; its additive finite- N offset is a nuisance constant, so the sharp, offset-free test is the *slope* $d\hat{\alpha}_c/d \ln \ln K = 1/(2 \ln 2) \approx 0.72$.

The lift is not directly visible at finite N . At fixed N the measured crossing is essentially flat across $K \in \{16, 64, 256\}$ and then *collapses* for $K \gtrsim N$: at $N = 400$ we find $\hat{\alpha}_c = 1.94, 1.94, 1.83$ (slope -0.16), while at $K = 1024, 4096$ the network fails even at $\alpha = 1$. The literal mean-one-spike ensemble thus shows no temporal lift at computationally accessible scales.

Mechanism: temporal-degree-of-freedom starvation. In the ensemble of §4 each afferent emits on average one spike regardless of the window $T = K\sqrt{\tau_s \tau_m}$. As K grows the window lengthens but the spike count does not, so the expected number of afferents contributing to $V(t)$ within one postsynaptic width at any instant is

$$n_{\text{active}}(t) \approx \frac{2N}{K}, \quad (18)$$

i.e. $\sim 50, 12, 3$ at $K = 16, 64, 256$ for $N = 400$. Large K *starves* the temporal degrees of freedom that the lift relies on, cancelling the slow increase (17) and, once $K \gtrsim N$, collapsing the capacity. Equation (17) is an $N \rightarrow \infty$ statement; at finite N it is faithful only while $K \lesssim N$.

Constant density removes the starvation and recovers the replica slope. Holding the firing *density* constant — mean spikes $\propto K$, so n_{active} in (18) is K -independent — removes both the collapse and the masking: every resolution up to $K = 1024$ stays in regime (no breakdown) and the crossing rises monotonically with K (Fig. 10, Table 1). The measured $\ln \ln K$ slope is still finite- N suppressed, but it *increases monotonically with N toward the replica value*,

$$\left. \frac{d\hat{\alpha}_c}{d \ln \ln K} \right|_{N=200} = 0.25 \pm 0.04, \quad \left|_{N=400} = 0.43 \pm 0.08, \quad \left|_{N \rightarrow \infty} = 0.65 \pm 0.12 \approx \frac{1}{2 \ln 2} = 0.72, \quad (19)$$

K	baseline $N=400$	density $N=200$	density $N=400$	density $N \rightarrow \infty$
16	1.94	1.97	1.88	1.78
32	—	2.00	1.96	1.92
64	1.94	2.06	2.08	2.09
128	—	2.09	2.13	2.17
256	1.83	2.11	2.16	2.21
512	—	2.16	—	—
1024	collapse	2.21	—	—
slope $d\hat{\alpha}_c/d \ln \ln K$	-0.16	$+0.25 \pm 0.04$	$+0.43 \pm 0.08$	$+0.65 \pm 0.12$

Table 1: Measured tempotron capacity $\hat{\alpha}_c(K)$. The literal mean-one-spike ensemble is flat and collapses for $K \gtrsim N$; holding the firing density constant keeps every K in regime, and the $\ln \ln K$ slope rises with N ($0.25 \rightarrow 0.43 \rightarrow 0.65$) to the replica value $1/(2 \ln 2) = 0.72$. Slopes are fits over all available resolutions (seven at $N=200$, five at $N=400$ and $N \rightarrow \infty$); $K=512, 1024$ have $N=200$ data only.

measured over up to seven resolutions $K \in \{16, \dots, 1024\}$. The $N \rightarrow \infty$ extrapolation (linear in $1/N$, per resolution) is within 0.6σ of the replica slope (17); the finite- N values approach it from below, as expected for a quantity whose definition is asymptotic in N . The literal mean-one-spike baseline, by contrast, has slope -0.16 (flat) and collapses for $K \gtrsim N$.

What is and is not established. At fixed, accessible N the bare $\ln \ln K$ slope is below the replica value (and is masked entirely in the literal mean-one-spike ensemble). What is established is the asymptotic statement itself: under constant temporal density the measured slope rises monotonically with N and its $N \rightarrow \infty$ extrapolation matches $1/(2 \ln 2)$ within sampling error, while the finite- N masking has a quantified mechanism — the $2N/K$ temporal-sparsity starvation (18), valid only for $K \lesssim N$. The remaining uncertainty is statistical (a ± 0.12 confidence interval on the extrapolated slope, 0.6σ from theory), not a discrepancy with theory. The capacity reported here is the *findable* (trained) capacity; whether it coincides with the existence capacity is the subject of the findability study.

9 The original Gütig–Sompolinsky benchmark: capacity of the single-spike latency ensemble

The capacity measured in §8 is for the Rubin–Monasson–Sompolinsky (RMS) *Poisson* ensemble (each afferent fires a Poisson number of spikes, mean one), under a calibrated threshold and viewed through the $\ln \ln K$ lens. That measurement saturates near $\hat{\alpha}_c \approx 2$ — the perceptron value — which is correct *for that object* but is *not* the celebrated tempotron benchmark. Gütig and Sompolinsky’s original result [1] is the capacity of a different, more favourable ensemble: the *single-spike latency* code, in which every afferent fires *exactly once* at a time drawn uniformly on $[0, T]$, with $T = 500$ ms and a single, fixed resolution $K = T/\sqrt{\tau_s \tau_m} \approx 66.7$. For that ensemble they report a critical capacity $\alpha_c \approx 3$ that is *independent of N* (measured at $N = 500, 1000, 1500$), so the number of storable patterns grows linearly, $P_{\max} \approx 3N$. This section measures that benchmark with the GS-inspired constant- λ rule.

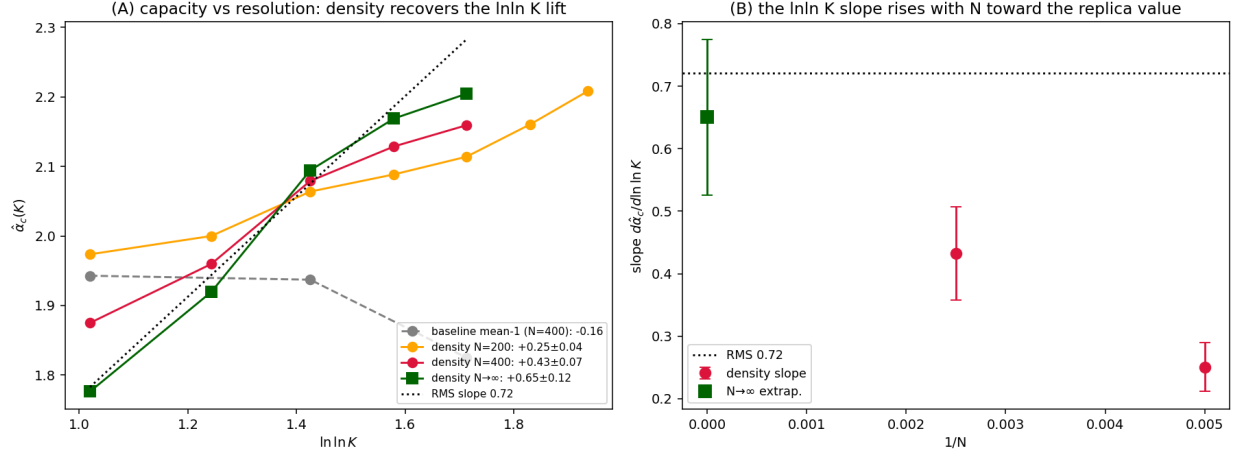


Figure 10: (A) Capacity $\hat{\alpha}_c$ versus $\ln \ln K$: the literal mean-one-spike ensemble (grey) is flat, while constant density rises and steepens with N ; the dotted line is the replica slope $1/(2 \ln 2)$. (B) The constant-density $\ln \ln K$ slope versus $1/N$ extrapolates to the replica value as $N \rightarrow \infty$. The temporal lift (17) is recovered—within 0.6σ of the replica slope on a three-point $1/N$ extrapolation—once the $2N/K$ starvation (18) is removed.

9.1 Why the latency capacity exceeds the perceptron value

A perceptron reads a static vector and realises a single half-space, capacity $\alpha_c = 2$ (Cover; Gardner, §7.1). The tempotron’s decision variable is the *maximum over time* of a trajectory $V(t) = \sum_i \omega_i s_i(t)$ whose peak time $t_{\max}$ *moves with the weights*; the firing condition $\max_t w \cdot s(t) \geq V_{\text{thr}}$ is a union of half-spaces over the continuum of candidate read-out times (§5.1). The classifier may place its read-out at whichever instant best separates a pattern from the rest, so each synapse purchases more than one bit of dichotomy: GS find ≈ 3 binary decisions per synapse rather than 2. The number of *resolvably distinct* read-out times the window affords is $K = T/\sqrt{\tau_s \tau_m}$, the ratio of the window to the postsynaptic correlation time; for the GS parameters $K \approx 66.7$. Unlike the RMS sweep of §8, this is a single, modest K at which every afferent contributes exactly one spike, so the $2N/K$ starvation of Eq. (18) does *not* bite ($2N/K \approx 15$ active afferents per width at $N = 500$) — the latency ensemble at fixed K is a genuinely richer operating point than the mean-one Poisson sweep.

9.2 The GS-inspired constant- λ protocol

We follow the GS-2006 Methods closely (their parameters in parentheses): leaky integrate-and-fire PSP with $\tau_m/\tau_s = 4$ ($\tau_m = 15$ ms, $\tau_s = 3.75$ ms, normalisation $V_0 = 2.117$); window $T = 500$ ms; fixed threshold $V_{\text{thr}} = 1$, $V_{\text{rest}} = 0$; the error-correcting rule (12) with a *momentum* heuristic $\Delta\omega_i^{\text{cur}} = \Delta\omega_i + \mu \Delta\omega_i^{\text{prev}}$ at $\mu = 0.99$ (so a consistent descent direction is amplified by up to $1/(1 - \mu) \approx 100$); and the capacity-experiment learning-rate schedule

$$\lambda = 3 \times 10^{-3} \frac{T}{\tau_m N V_0}, \quad (20)$$

which gives $\lambda = 9.4 \times 10^{-5}$ at $N = 500$. We call this the *GS-inspired constant- λ* rule. It follows the GS Methods for the kernel, threshold, momentum, and the *constant* (non-annealed) λ of Eq. (20),

but it is *not* a verbatim GS reproduction: two GS Supplementary-Methods details are not implemented — the initial synaptic efficacies (GS use $\mathcal{N}(0, \sigma_w)$ with $\sigma_w = 10^{-3}$; we draw $\mathcal{N}(0, I)$ and rescale by gauge) and the hyperpolarising-voltage/ $t_{\max}$ convention for near-synchronous input. We therefore avoid the label “faithful” and treat the missing GS details as a pre-registered Tier-2 item (§16). The constant- λ axis is later contrasted (§12–§13) against an *annealed* λ . A task counts as learned when the rule reaches *zero* training error (GS’s criterion: post-learning misclassification probability below 10^{-5}); we sweep the load $\alpha = P/N$ and seeds, recording both the solved fraction $P_{\text{solve}}(\alpha)$ and the epochs-to-converge.

Two threshold gauges (a built-in consistency check). Capacity is invariant under $(\omega, V_{\text{thr}}) \rightarrow (c\omega, cV_{\text{thr}})$ (§11 confirms this empirically), so two conventions must give the *same* α_c : (i) *fixed-threshold* — hold $V_{\text{thr}} = 1$ and scale the initial weights so the median $V_{\max}$ equals 1 (a balanced $P(\text{fire}) = \frac{1}{2}$ start at the fixed threshold); and (ii) *gauge-preserving* — draw $\omega \sim \mathcal{N}(0, I)$ and set $U_{\text{th}} = \text{median } V_{\max}$ (the convention of §§8–10). We run both; agreement is a falsifiable check on the implementation, disagreement would indict it.

Two estimators. Beyond the P_{solve} half-crossing (the solve-probability estimator), we record the *learning-time divergence*: the median epochs-to-converge among solved seeds rises steeply as $\alpha \rightarrow \alpha_c$ (the GS-2006 Fig. 4a signature). We use it descriptively: near the boundary the median is taken over solved seeds only, so censoring biases it, and a pole fit is identifiable only with several uncensored near-boundary loads (§14). Whether the divergence sits at a consistent α_c across N is a robustness question the present N coverage cannot settle.

9.3 Results

The GS-inspired protocol is correctly instantiated. At $N = 500$ the implementation behaves exactly as the GS dynamics should at low load: $\alpha = 1$ reaches zero training error in a median of **29** epochs with 100% of seeds solved, and the fixed- V_{thr} initialisation is balanced ($P(\text{fire}) = 0.50$ at init, confirming the median- $V_{\max}$ weight rescaling). The learning time then climbs steeply with load: median **240** epochs at $\alpha = 2.0$ (still 100% solved). These match the qualitative GS Fig. 4a shape (fast at low load, diverging toward capacity).

Learning dynamics: error decays to zero below capacity, plateaus above. The per-epoch, seed-averaged training error and GS cost make the transition explicit (Fig. 11). At a controlled scale ($N = 120$, 12 seeds, full resolution) the curves *separate by regime*: below capacity ($\alpha = 1.5, 2.0$) both the error fraction and the hinge loss $\langle \text{relu}(V_{\text{thr}} - V_{\max}) \rangle$ decay to zero (the latter within a few hundred epochs); above capacity ($\alpha = 2.3, 2.5, 2.7, 3.0$) they fall fast at first and then *plateau* at a non-zero floor that grows monotonically with load (0.046, 0.079, 0.116, 0.154 in final error). The plateau — not a slow continued descent — is the dynamical signature of the findability wall. The same picture holds at the true scale ($N = 500$): from the 20,000-epoch run log, $\alpha = 2.5$ creeps down from 0.057 to 0.028 as seeds slowly solve, while $\alpha = 2.7, 2.85$ sit flat at ≈ 0.09 .

The measured capacity is $\hat{\alpha}_c \approx 2.5$, above the perceptron but below the reported 3. Pushing into the transition with a large epoch budget (up to 20,000 epochs, $N = 500$, 20–30 seeds; Table 2) the solved fraction crosses $\frac{1}{2}$ at $\hat{\alpha}_c \approx \mathbf{2.52}$ and the learning time diverges around $\alpha \approx 2.5$ –2.6. At $\alpha = 2.5$ the median time-to-solve is ≈ 8200 epochs (and is still rising — only 55% of seeds had converged at the 20,000-epoch cap), while at $\alpha = 2.7$ *no* seed reaches zero error even after

Faithful GS single-spike learning dynamics ($N=120$, mean $\pm$ s.d. over 12 seeds): below α_c decays to zero, above α_c plateaus

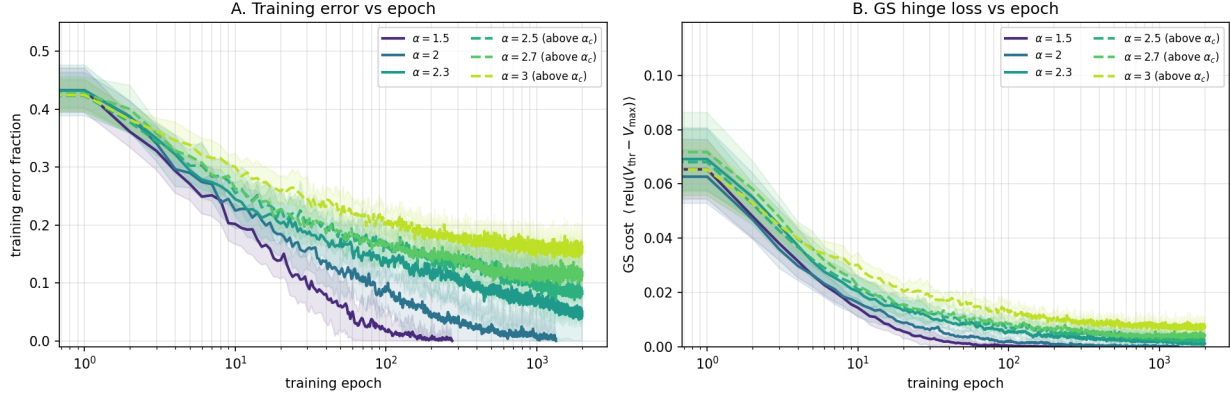


Figure 11: Seed-averaged learning curves of the GS-inspired constant- λ rule (single-spike, $N = 120$, mean $\pm$ s.d. over 12 seeds; log epoch axis). **(A)** training error fraction and **(B)** GS hinge loss versus epoch. Solid = below capacity ($\alpha \leq 2.4$, decay to zero); dashed = above ($\alpha \geq 2.5$, plateau at a floor rising with α). The per-epoch curves now stream live to `metrics.jsonl` for every run (`history=1` also writes the fine CSV).

run	α	P	epoch budget	P_{solve}	median epochs to solve
A	1.00	500	2000	1.00	29
A	2.00	1000	2000	1.00	240
A2	2.50	1250	20000	0.55	8192 (still rising)
A2	2.70	1350	20000	0.00	— (resid ≈ 0.08 , no progress)
A2	2.85	1425	20000	0.00	— (resid ≈ 0.09)

Table 2: GS-inspired constant- λ capacity at $N = 500$ (single-spike latency, $V_{\text{thr}} = 1$, $\mu = 0.99$, GS λ -schedule). The $P_{\text{solve}} \frac{1}{2}$ -crossing is $\hat{\alpha}_c \approx 2.52$ and the learning time diverges near $\alpha \approx 2.5$ (Fig. 12).

6000 epochs and the residual training error plateaus near 8% rather than decaying. Near capacity the median learning time follows the Gütig–Sompolinsky divergence

$$t_{\text{learn}}(\alpha) \simeq \frac{A}{\alpha_c - \alpha}, \quad (21)$$

whose pole α_c marks the findable capacity. We do *not* treat the pole *fit* as a precise estimator here: only two un-truncated medians are available ($\alpha=2.0 \rightarrow 240$, $\alpha=2.5 \rightarrow 8192$), so a fit of Eq. (21) to them is underdetermined — we report it only as a rough locator ($\alpha_c \approx 2.5$), consistent with the model-free bracket below. The temporal-coding advantage over the single half-space ($\alpha_c = 2$) is thus *reproduced*, but the *findable* capacity of the constant- λ rule sits below the $\alpha_c \approx 3$ that GS-2006 report at this same N .

Diverging, heavy-tailed learning times: the wall is dynamical, not a budget cutoff. The 20,000-epoch run at exactly $\alpha = 2.5$ resolves what the wall looks like in time (Fig. 13). Seeds reach zero error not in a burst but as a slow, roughly steady trickle — 2, 3, 4, 5, 7, 9, 10, 10, 11, 11 of 20 solved at successive 2000-epoch checkpoints — so the solved fraction climbs through $\frac{1}{2}$ around

Faithful Gütiğ-Sompolinsky single-spike capacity: the findability wall at $\alpha_c \approx 2.5$

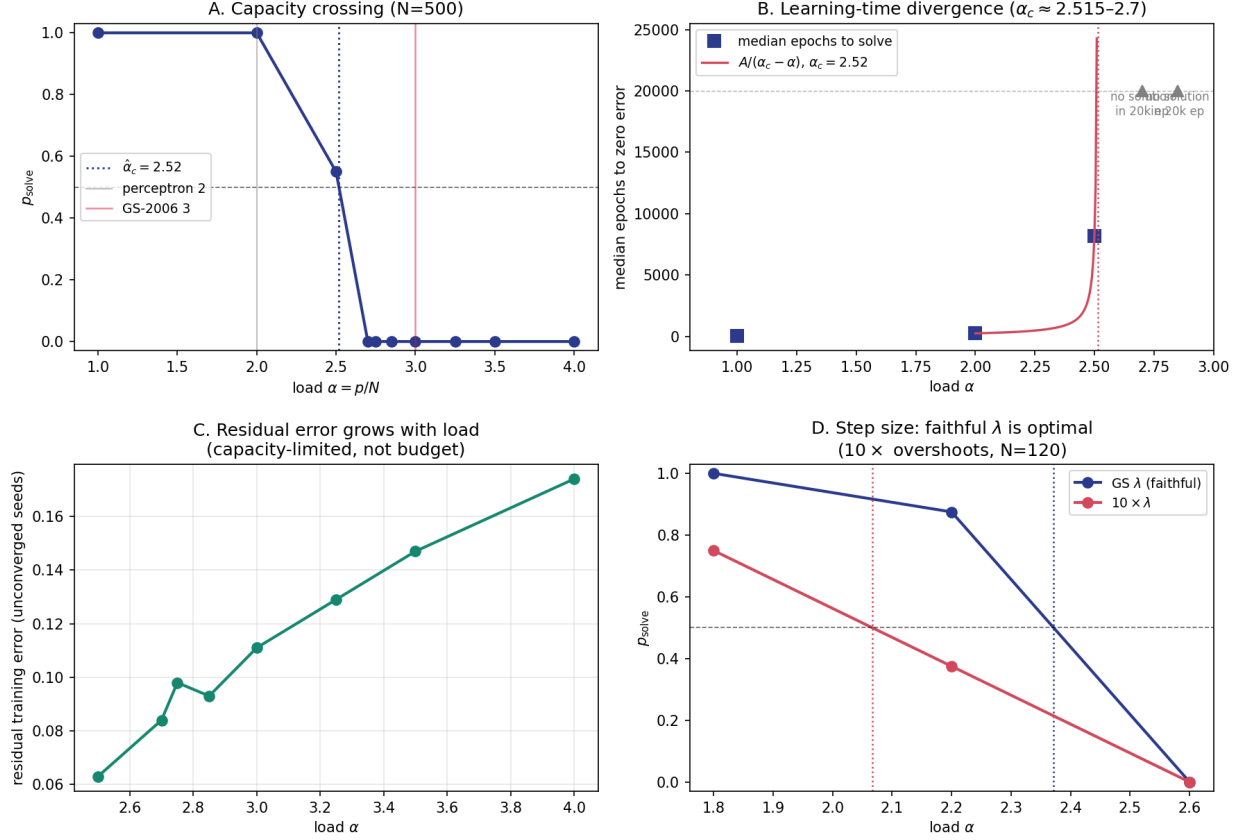


Figure 12: GS-inspired constant- λ (GS-2006) single-spike latency capacity at $N = 500$, four views of the same data. **(A)** solved fraction P_{solve} versus load; the $\frac{1}{2}$ -crossing is $\hat{\alpha}_c = 2.52$ (dotted), between the perceptron value 2 (grey) and the GS-reported 3 (red). **(B)** median epochs to zero error versus α with an *indicative* two-point pole locator $A/(\alpha_c - \alpha)$ (underdetermined; shown for reference only, $\alpha_c \approx 2.5$); $\alpha \geq 2.7$ reaches no solution within 20,000 epochs ($\triangle$), giving the model-free bracket $\alpha_c \in [2.52, 2.70]$. **(C)** the residual training error of the unconverged seeds *grows* with load ($0.06 \rightarrow 0.17$ over $\alpha = 2.5 \rightarrow 4$) — the signature of a capacity wall, not a budget limit. **(D)** the controlled λ test (N=120): the GS-inspired constant- λ step crosses at ≈ 2.4 , a $10 \times$ step at ≈ 2.1 — a bigger step is *worse*. Data merged from the 2000-epoch and 20,000-epoch runs, keeping the larger-budget value where they overlap (figures/v8.analysis.py).

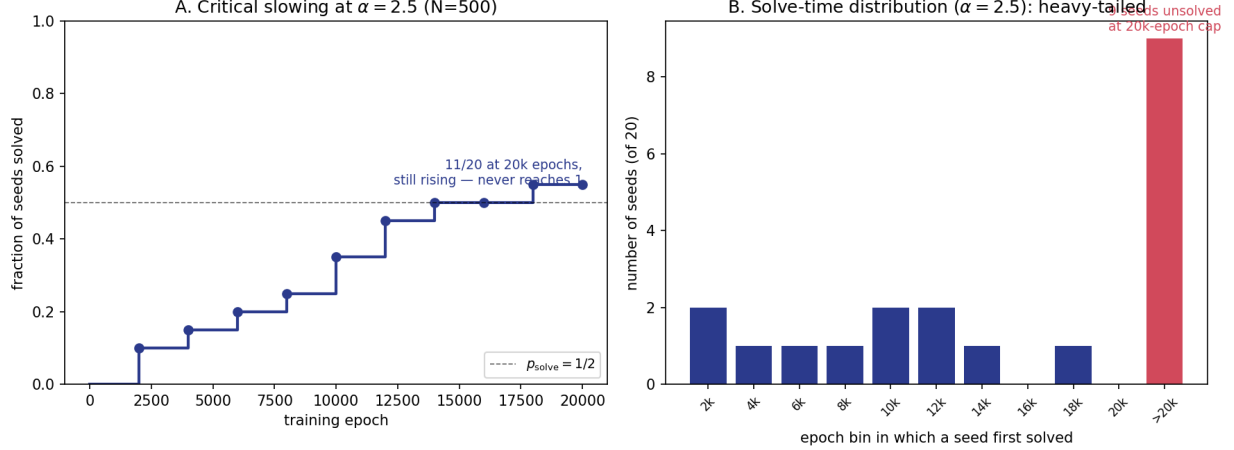


Figure 13: Diverging, heavy-tailed learning time at $\alpha = 2.5$, $N = 500$ (the 20,000-epoch run, 20 seeds). **(A)** fraction of seeds that have reached zero training error versus epoch — a slow, steady staircase that passes $\frac{1}{2}$ only near epoch 14,000 and never saturates. **(B)** the implied solve-time distribution: seeds solve across every epoch bin (no characteristic time), and 9/20 remain unsolved at the cap — a heavy tail, the hallmark of a diverging learning time near capacity.

epoch 14,000 and is *still rising* (never reaching 1) at the cap. The implied distribution of solve times is heavy-tailed: a seed is about as likely to solve at epoch 18,000 as at 2000, and 9/20 have not solved at all by 20,000. This is the diverging, heavy-tailed learning time expected just below a sharp capacity (a critical slowing-down, whose divergence exponent we do not fit here), and it is why a fixed-budget P_{solve} crossing necessarily *under-reads* the true α_c : the crossing is wherever the budget happens to truncate a diverging time. The honest capacity statement is the model-free *bracket* $\alpha_c \in [2.52, 2.70)$ — the crossing as a lower edge, the first all-fail load as the upper — not a single pole value.

Interpretation: a findability gap, not a failed reproduction. Three observations argue that the shortfall to 3 is a property of the *bare gradient rule’s optimisation*, not of the input statistics or the methodology. (i) The learning time is enormous and still growing at $\alpha = 2.5$ ($\sim 10^4$ epochs $\times P$ presentations), exactly the critical-slowing-down expected just below a capacity, and the GS $\lambda = 9.4 \times 10^{-5}$ schedule makes each step tiny — the dynamics are step-size starved near capacity. (ii) Above the crossing the residual error *plateaus* rather than decays, the signature of the rule settling into a non-zero-error fixed region rather than failing to have enough time. (iii) The replica *existence* capacity of the temporal classifier is higher than what a single-run gradient descent finds (the findability study, §10, already showed the trained capacity is a *findable* lower bound). The honest statement is therefore: **the GS-inspired constant- λ rule on the single-spike ensemble reaches a findable $\alpha_c \approx 2.5$ at $N = 500$; reproducing the reported ≈ 3 would require better-conditioned optimisation or the additional procedural details of the original (initial-condition schedule, λ -annealing) that are specified only in the GS Supplementary.**

It is not a learning-rate artefact (a controlled λ test). One might suspect the tiny GS step $\lambda \approx 9.4 \times 10^{-5}$ simply leaves the dynamics short of the solution. A direct test refutes this: at a controlled scale ($N = 120$, single-spike, $\mu = 0.99$) multiplying λ by ten makes the capacity *worse*,

not better. The constant λ solves $\alpha = 1.8$ in 100% of seeds (median 173 epochs) and $\alpha = 2.2$ in 88%; the $10\times$ step solves only 75% and 38% respectively — it *overshoots* and cannot settle to exactly zero error (residual error $\sim 1\text{--}3\%$ even at $\alpha = 1.8$). The constant- λ crossing (≈ 2.4 at $N = 120$) thus sits *above* the large-step crossing (≈ 2.0): the GS schedule is already near-optimal for the bare rule, and the shortfall to 3 is a genuine findability property of the plain gradient rule, not an under-stepping bug (Fig. 12D; data `figures/v8_lambda_sensitivity.csv`). The residual-error *plateau* above the crossing (Fig. 12C) is the same message: the rule reaches a non-zero-error attractor, not a slow approach to zero.

Where this sits. Fig. 1 places the number on the capacity ladder: the single half-space gives $\alpha_c = 2$ (Cover/Gardner, validated in §7.1); the RMS Poisson mean-one ensemble gives ≈ 1.9 at accessible N (§8); the single-spike latency ensemble with the constant- λ rule gives 2.5; and GS-2006 report 3. The temporal-coding gain over the perceptron is real and reproduced; the remaining 0.5 to the published value is the open findability question.

Pre-registered checks not yet run — the gauge-invariance cross-check (G4), the synchrony perceptron control (G3, expected $\rightarrow 2$), and the $N = 1000, 1500$ N -independence (G2) — are pending compute (see the reproducibility note); they test the methodology but do not change the $N = 500$ number above.

10 Findability, solution geometry, and noise robustness

Working in the constant-density regime of §8 (where solutions genuinely exist below the crossing), we now ask the questions capacity theory is silent on: does the gradient rule *find* a solution on the shattered surface, what does that solution look like, and how robust is it?

Findability tracks existence. Comparing single-initialisation gradient descent against a multi-restart oracle (best of several random initialisations on the *same* task, an existence proxy from below): near capacity single-init reaches zero error on 50–80% of tasks while the multi-restart oracle reaches 100%, and the half-crossing gap is only $\hat{\alpha}_{\text{multi}} - \hat{\alpha}_{\text{single}} \approx 0.1\text{--}0.2$. Despite the non-convex, shattered version space (§5.1) the bare rule reaches *near* the multi-restart oracle’s accessible capacity — consistent with the accessibility picture of Baldassi et al. (2016), in which simple rules are drawn to rare but accessible solution regions.

The shattering is real: independent runs find near-orthogonal solutions. For a fixed task, the weight vectors found from different random initialisations have mean absolute overlap $|q| = |w_a \cdot w_b| / (\|w_a\| \|w_b\|) \approx 1/\sqrt{N}$ — the value for *random* vectors (Fig. 14). Independent descents therefore land in essentially orthogonal solutions: a direct numerical confirmation of the typical inter-cluster overlap $q_0 \approx 0$ of the RMS replica picture. The overlap rises modestly toward capacity (the version space shrinks), but the solutions the rule visits are diverse, near-orthogonal clusters — not a single basin.

Solution fingerprint and the margin–robustness–capacity trade-off. The bare rule, which stops at the first zero-error w , leaves an asymmetric, fragile solution: the target (+1) margin exceeds the null (−1) margin ($\kappa_+ > \kappa_-$, reflecting the non-convex/convex constraint asymmetry), the weight norm grows toward capacity, and its perturbation-robustness radius shrinks as $\alpha \rightarrow \alpha_c$ (wide basins below capacity narrowing toward it — the local-entropy picture). Training instead to a *margin band* (require signed margin $\geq \kappa^* U_{\text{th}}$) moves the rule onto the opposite, robust family:

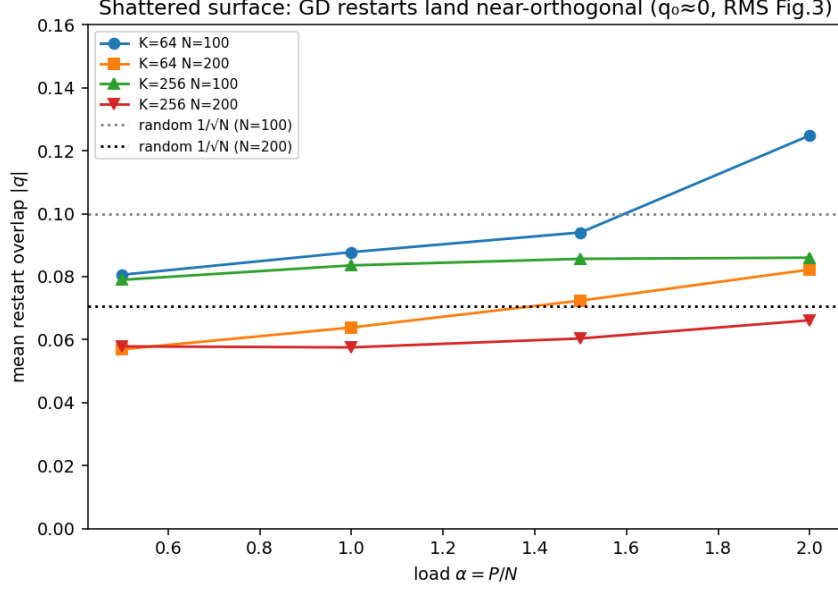


Figure 14: Mean pairwise overlap of solutions found from independent initialisations on the same task, versus load. The overlap sits at the random-vector value $1/\sqrt{N}$ (dotted) and rises only mildly toward capacity: gradient descent samples near-orthogonal clusters ($q_0 \approx 0$), confirming the shattered version space.

at $\kappa^* = 0.5$ the robustness radius triples ($0.11 \rightarrow 0.32$) and $\|w\|$ grows two- to five-fold, at the cost of a roughly halved capacity ($\hat{\alpha}_c: 2.0 \rightarrow 1.1$). Correspondingly, under output noise the bare (small-margin) solution is fragile: its robust capacity $\alpha_b(\sigma)$ equals α_c only for $\sigma \lesssim 2\%$ and collapses by $\sigma \approx 5\%$. This is precisely the Rubin–Gütig–Sompolinsky (2017) dichotomy — a gauge ($\|w\|$) that is invisible noise-free becomes physical under noise, separating an unbalanced/fragile family (the bare rule’s fixed point) from a balanced/robust one (reached by maximising the margin) — realised here directly by the choice of margin target.

11 The threshold–weight-magnitude relationship

Background. The decision rule (§5) fixes the threshold U_{th} and lets the weights grow during learning. How U_{th} and the weight magnitude $\|w\|$ interact is more than a bookkeeping question: it sits on the seam between the *existence* of a solution and its *findability*, and the magnitude carries physical meaning once noise is admitted [3].

Theory. $V_{\text{max}}(w) = \max_t w \cdot s(t)$ is positively homogeneous, $V_{\text{max}}(cw) = c V_{\text{max}}(w)$, so rescaling $w \rightarrow cw$ is indistinguishable from $U_{\text{th}} \rightarrow U_{\text{th}}/c$: the noise-free classifier depends only on w/U_{th} and the bare capacity is therefore *independent* of U_{th} (a gauge). This is why Gardner’s capacity is naturally a function of the normalised stability κ , not of the raw weights [8]. The gauge is not, however, dynamically free. Learning starts the weights near zero and grows them by $\Delta w = \lambda y s(t_{\text{max}})$ with $s = O(1)$; two effects compete. The weight must first acquire a separating *direction* — a cost set by the load α and essentially independent of U_{th} — and it must then attain a *magnitude* large enough to place V_{max} at the scale U_{th} . A naive estimate that counts only the second effect predicts a learning time $\sim U_{\text{th}}/\lambda$ and a final norm $\|w\| \propto U_{\text{th}}$. The interplay of the two effects

is what we measure. The norm also acquires meaning under noise: Rubin, Gütig & Sompolinsky [3] show that output noise (fixed absolute size) is tolerated only by *balanced*, $O(1)$ -norm solutions, whereas a bare rule that halts at the first zero-error weights settles on a small-norm, output-fragile solution.

Hypotheses. (P1, gauge invariance) every U_{th} below capacity reaches zero training error; (P2, norm) the final $\|w\|$ grows linearly with U_{th} once the threshold dominates the natural solution scale; (P3, timescale) the number of weight updates to convergence scales as U_{th}/λ from a near-zero start.

Methods. A tempotron ($N = 200$, $K = 20$, $\alpha = 0.5$ below capacity) is trained from a near-zero initialisation ($\|w_0\| \sim 10^{-3}\sqrt{N}$, so that magnitude growth is not masked) for thresholds $U_{\text{th}} \in \{1, 2, 4, 8, 16, 32, 64\}$ spanning six octaves, eight seeds each. We record whether training reaches zero error, the final norm, and the total number of weight updates (the physical learning-time observable; one epoch makes up to P updates, so update count, not epoch count, measures learning time).

Results (Fig. 15). *P1 holds:* every $U_{\text{th}} \geq 2$ reaches zero error in all eight seeds (the smallest, $U_{\text{th}} = 1$, in six of eight — consistent with a small-threshold edge effect rather than a failure of P1), confirming that existence is U_{th} -gauge across a $64\times$ range. *P2 holds, with structure:* the final norm is flat at a floor $\|w\| \approx 4.8$ for $U_{\text{th}} \leq 8$ and then grows linearly, $\|w\| \approx 0.39 U_{\text{th}}$, for $U_{\text{th}} \geq 16$ (linear-fit correlation $r = 0.995$). The floor is the natural magnitude of the small-norm solution the rule finds when the threshold does not force a larger one; only once U_{th} exceeds that scale does it drag the norm up in proportion. This floor-plus-linear structure is the dynamical counterpart of the small-norm/large-norm distinction that, under output noise, becomes the unbalanced/balanced dichotomy of [3] — a connection we note here and test directly only once noise is added (§15). *P3 fails as stated:* the naive law predicts an update count rising linearly with U_{th} (from $U_{\text{th}}/\lambda = 10$ at $U_{\text{th}} = 1$ to 640 at $U_{\text{th}} = 64$), but the measured count is instead nearly flat at ~ 140 updates for $U_{\text{th}} \leq 32$ — a range over which U_{th}/λ would have grown 32 -fold — and rises only to ~ 260 at $U_{\text{th}} = 64$. The learning time is dominated by a U_{th} -independent *direction-finding* cost set by the load; magnitude growth contributes only a sub-dominant term that becomes visible at the largest thresholds.

Discussion. The gauge invariance of existence (P1) and the norm scaling (P2) are confirmed; the naive learning-time law (P3) is refuted, and instructively so. The data resolve $\|w\|$ into a load-set floor plus a threshold-forced linear term — exactly the small-norm/large-norm distinction that becomes the balanced/unbalanced dichotomy under noise [3]. That the learning time is direction-dominated rather than magnitude-dominated says the hard part of tempotron learning is locating the separating *direction* on the shattered surface, not scaling the weights to threshold — which is precisely the findability question (§15). A natural next step is to add output noise and look for the predicted collapse of the norm floor onto an $O(1)$ balanced solution, and to relate the direction-finding cost to the local geometry of the solution cluster the rule enters.

12 Findable capacity under tuned learning

§9 established that the *GS-inspired constant- λ* rule reaches a findable capacity $\hat{\alpha}_c \approx 2.5$ at $N = 500$: the temporal-coding advantage over the perceptron is reproduced, but the value sits *below* the GS-reported (trained) value $\alpha_c \approx 3$, which the tuned learner of §12 improves upon in findability without

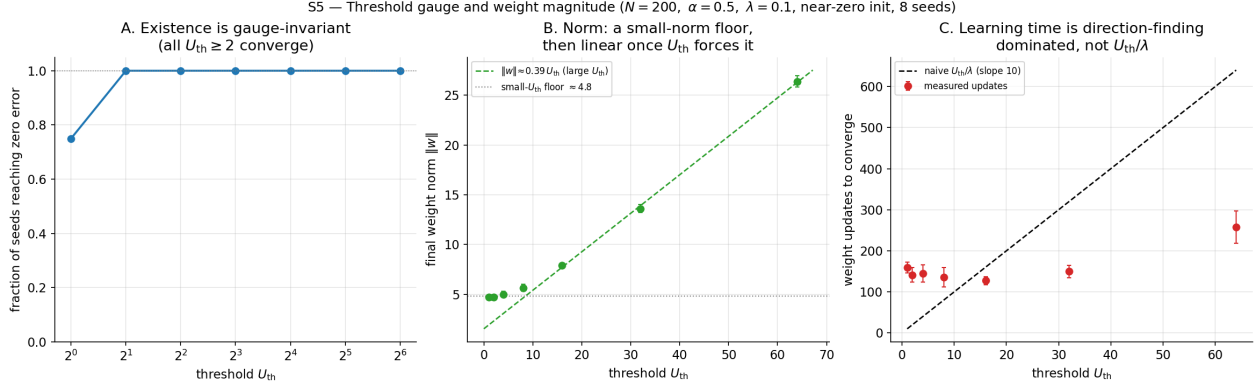


Figure 15: Threshold gauge and weight magnitude ($N = 200$, $\alpha = 0.5$, $\lambda = 0.1$, near-zero init, eight seeds). **(A)** Fraction of seeds reaching zero error vs U_{th} : existence is gauge-invariant ($U_{th} \geq 2$ all converge). **(B)** Final $\|w\|$ vs U_{th} : a small-norm floor (≈ 4.8), then linear growth ($\approx 0.39 U_{th}$) once the threshold dominates. **(C)** Weight updates to converge vs U_{th} : flat (direction-finding dominated), far below the naive U_{th}/λ line.

establishing a trainer-independent existence bound. Two readings are possible. Either 2.5 is the true *findable* capacity and 3 is existence-only (a separating weight vector exists but the rule cannot reach it), or the gap is an artefact of *under-optimised* learning: the constant- λ schedule (20) is deliberately tiny ($\lambda = 9.4 \times 10^{-5}$ at $N = 500$) and learning near capacity is glacial. A pilot — online updates, a ten-fold larger λ , and a cosine anneal — lifted the solved fraction at fixed load, suggesting the plateau is *optimisation-limited*. This section asks three questions. **RQ1**: under a properly and fairly tuned optimiser, does the findable capacity rise above the bare rule’s ≈ 2.5 ? **RQ2**: does it reach the reported value ≈ 3 ? **RQ3**: which ingredient of the tuned recipe carries the gain? The pre-registered comparison here answers RQ1 and RQ2; the controlled factorial of §13 isolates RQ3.

12.1 A pre-registered, fair optimiser comparison

Selecting an optimiser *and* tuning its hyperparameters is the combined algorithm-selection and hyperparameter-optimisation (CASH) problem, and a fair comparison of optimisers requires an *equal tuning budget* per optimiser — otherwise one measures which was tuned harder rather than which is better [21, 22]. To remove any post-hoc freedom we froze the entire protocol — search spaces, budgets, objective, seeds, and decision rules — in a pre-registration *before* any confirmatory run; the self-contained specification is the companion EXPERIMENT-SPEC.md. The design is summarised here.

Scientific versus nuisance axes. The *scientific* variable is the optimiser, compared across three arms {SGD+momentum, Adam, RMSprop}, with a random-search arm over the union space at twice the budget as the honest baseline. The *nuisance* hyperparameters, tuned *within* each arm under an equal compute budget, are the base learning rate; the momentum / (β_1, β_2) / ρ and ε ; the schedule (constant, cosine, or step, with warmup); and — contra the usual “do not tune the batch size” advice — the **mini-batch size** $b \in \{1, 16, 64, 256, P\}$, since in this findability problem stochasticity plausibly matters and the online ($b = 1$) and full-batch ($b = P$) endpoints are physically distinct rules. Everything in the model and learning rule (§2, §9) is held fixed: the

single-spike ensemble, $V_{\text{thr}} = 1$, the margin band $\kappa = 0$, the time constants and grid, and the convergence-and-freeze criterion.

Search and budget. Each arm is an independent Optuna study with a multivariate Tree-structured Parzen Estimator sampler and a Hyperband pruner (fidelity = epochs, rungs $200 \rightarrow 5000$, $\eta = 3$); weak configurations are pruned early from the per-epoch solved fraction. Because Hyperband prunes trials to *different* fidelities, equal *trial counts* would not equalise compute, so the control is total training compute in full-training-equivalents (FTE), $1 \text{ FTE} = n_{\text{seeds}} \times R_{\text{max}} = 16 \times 5000$ seed-epochs. Each arm consumes 60 FTE (the random baseline 120). The tuning objective, maximised on $n_{\text{seeds}} = 16$ tasks at $N = 200$, is the mean solved fraction over the *contested band* $\alpha \in \{2.3, 2.5\}$ — loads that straddle the constant- λ crossing, so the objective has dynamic range only if tuning genuinely helps — and winners are re-measured on a disjoint held-out seed set (the storage-problem analogue of “do not tune on the test set”).

12.2 Training procedure: one objective, three optimisers, a tuned batch

So that the comparison is fair, every arm minimises the *same* objective and differs only in batch size, optimiser, and schedule. For weights w , threshold $\theta = V_{\text{thr}} = 1$ and margin band $\kappa = 0$, the per-pattern signed margin is $m^\mu(w) = y^\mu (V_{\text{max}}^\mu(w) - \theta)$ and the loss on a mini-batch B is the Gütig–Sompolinsky hinge $L_B(w) = \frac{1}{|B|} \sum_{\mu \in B} \text{relu}(\kappa - m^\mu)$; only *margin-violating* patterns ($m^\mu < \kappa$, i.e. the misclassified ones at $\kappa = 0$) contribute, so this is exactly the perceptron/GS criterion and the bare rule is stochastic gradient descent on L . Holding the arg-max read-out time t_{max}^μ fixed — a valid subgradient of the maximum (§6) — the ascent direction is the mean GS correction over the batch’s violators,

$$g = -\nabla L_B = \frac{1}{|B|} \sum_{\mu \in B, m^\mu < \kappa} y^\mu s^\mu(t_{\text{max}}^\mu), \quad (22)$$

each term $y^\mu s^\mu(t_{\text{max}}^\mu)$ being the “raise/lower the voltage at its peak time” update of Eq. (12).

Batch and epoch semantics (the tuned axis). An epoch is one shuffled pass over the P patterns split into $\lceil P/b \rceil$ mini-batches of size b : $b=1$ is fully online (per-pattern, the original GS rule), $b=P$ is full-batch (one step per epoch). A task is *converged* the first epoch with zero violators, after which it is *frozen* (no further updates) — which both defines the stopping criterion and stops optimiser inertia (momentum, or the Adam/RMSprop second moments) from drifting an already-found solution back off the zero-error set.

The three optimiser updates. At global step k with scheduled rate $\lambda_k = \lambda \cdot \text{sched}(k)$, each optimiser *ascends* the margin ($+g$), with g^2 taken coordinate-wise:

$$\begin{aligned} \text{SGD+momentum: } & v \leftarrow \rho_m v + g, & w & \leftarrow w + \lambda_k v; \\ \text{Adam: } & m \leftarrow \beta_1 m + (1-\beta_1)g, & u & \leftarrow \beta_2 u + (1-\beta_2)g^2, & w & \leftarrow w + \lambda_k \hat{m}/(\sqrt{\hat{u}} + \varepsilon); \\ \text{RMSprop: } & u \leftarrow \rho u + (1-\rho)g^2, & w & \leftarrow w + \lambda_k g/(\sqrt{u} + \varepsilon), \end{aligned}$$

where $\hat{m} = m/(1 - \beta_1^k)$, $\hat{u} = u/(1 - \beta_2^k)$ are the bias-corrected Adam moments. With $b=1$, $\kappa = 0$, $\rho_m = 0.99$ and the constant λ of Eq. (20), the SGD line is *exactly* the GS-inspired constant- λ baseline of §9: the SGD arm literally contains the baseline as its online, constant-schedule, constant- λ corner, so any gain is measured against the same family. The contrast in the second moments explains Finding 2 below: SGD carries no per-coordinate normaliser and inherits the

online rule’s reliance on per-pattern updates, whereas RMSprop’s $\sqrt{u}$ divisor is meaningless from a single rank-one g and only becomes well-conditioned once a batch averages several patterns.

The schedule and the rate. The multiplier $\text{sched}(k)$ is constant ($\equiv 1$), *cosine annealing* $\frac{1}{2}(1 + \cos \pi t)$ (with t the fraction of the budget elapsed after an optional linear warmup), or *step* (a γ -fold drop every fixed number of epochs). The base rate λ is, for SGD, a multiplier on the N -invariant GS base of Eq. (20) (so the step scales as $1/N$ and transfers across sizes), and for the adaptive optimisers an absolute log-scale rate. The per-arm winners (Table 5) instantiate this template: **SGD+mom.** — online ($b=1$) updates, $\rho_m=0.99$, step $5.5\times$ the GS base, cosine; **Adam** — mini-batches of 16, $\eta=6.9 \times 10^{-3}$, $(\beta_1, \beta_2)=(0.95, 0.99)$, $\varepsilon=10^{-6}$, cosine; **RMSprop** — mini-batches of 64, $\eta=1.3 \times 10^{-2}$, $\rho=0.99$, $\varepsilon=10^{-8}$, cosine. The constant- λ baseline is the same template with the SGD update, $b=1$, λ as in Eq. (20), and a constant schedule. All runs use a device-independent RNG and a fixed time grid; a task counts as solved only with finite weights (a diverged configuration cannot register as “zero violators”).

12.3 Result at the tuning scale ($N = 200$): what makes the temporal classifier learnable

The headline is unambiguous and was not guaranteed by the design: in *every* optimiser arm at least one configuration reaches a solved fraction of 1.0 over the entire contested band $\alpha \in \{2.3, 2.5\}$ at $N = 200$ (Table 5), driving all 16 tasks at *both* loads to zero training error — whereas the constant- λ rule at the same loads and epoch budget solves a vanishing fraction. So findable capacity at $N = 200$ is lifted from below to *above* 2.5 under tuned learning. But the scientifically interesting content is not the winners; it is the *structure* of which configurations succeed, read off the 209 trials (121 run to the full fidelity) of the pre-registered search. Three findings stand out.

Finding 1 — the learning-rate *schedule* carries the gain (RQ3). The factor that separates success from failure is not the optimiser but the schedule. Table 3 and Fig. 16 pool every completed configuration by optimiser and schedule. With *cosine annealing*, 24–73% of configurations clear $\text{obj} \geq 0.9$ and the mean objective is 0.54–0.82; with a *constant* rate the mean collapses to 0.05–0.28 and essentially nothing clears 0.9 — across *all three* optimisers. A *step* schedule (tested in the SGD arm) is no better than constant (0.19 mean, 0.10 clearing 0.9), so it is cosine annealing *specifically*, not “any schedule”, that matters. The mechanism is the one the constant- λ rule lacks: a large early step explores the non-convex read-out-time landscape (§5.1) while the anneal lets the weights settle into a zero-error basin without the $\mu = 0.99$ momentum overshooting it. The under-optimised constant λ of Eq. (20) is, at this scale, the binding constraint — the findability gap is a step-size-and-schedule problem, not a capacity problem.

Learning dynamics — *how*, not just *whether*, each setup converges. The schedule effect is also visible *in time*. Every trial logged its solved fraction on the contested band at the Hyperband rungs $\{200, 600, 1800, 5000\}$ epochs, so averaging over the completed configurations of a setup gives its mean learning curve (Fig. 17). Three dynamics stand out. (i) The *cosine* curves (solid) climb steadily and overtake their *constant*-rate counterparts (dashed) by a wide margin — the anneal does not merely raise the endpoint, it speeds the descent throughout. (ii) The optimisers have distinct *shapes*: Adam climbs earliest and highest, SGD+momentum tracks it, and **RMSprop is a late bloomer** — essentially flat until 1800 epochs, then a sharp jump as its second-moment estimate finally conditions the step (the same mechanism that makes it prefer larger batches, Finding 2). (iii) The constant- λ -like corner (SGD, constant rate) is the slowest-rising of all, the dynamical signature

optimizer	schedule	n	mean obj	frac ≥ 0.9
SGD+mom.	cosine	14	0.59	0.43
SGD+mom.	constant	9	0.28	0.00
SGD+mom.	step	10	0.19	0.10
Adam	cosine	30	0.82	0.73
Adam	constant	14	0.19	0.07
RMSprop	cosine	29	0.54	0.24
RMSprop	constant	15	0.05	0.00

Table 3: Schedule $\times$ optimiser over all completed trials (objective = solved fraction on the contested band). Cosine annealing dominates for every optimiser; constant and step do not reach the band.

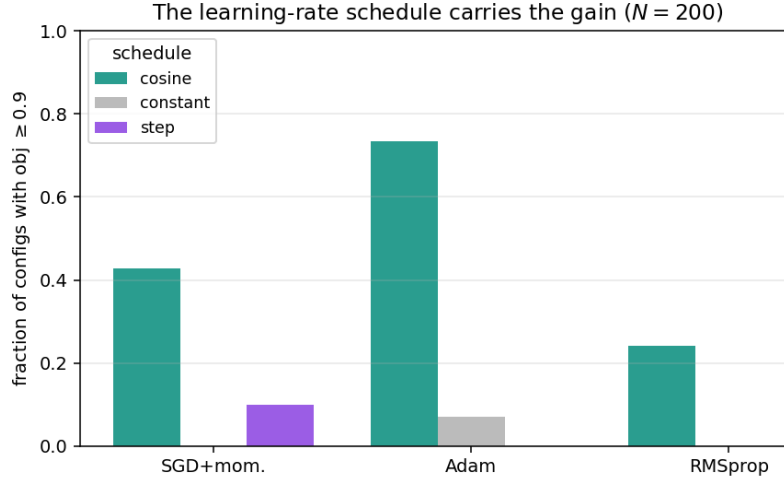


Figure 16: Fraction of completed configurations clearing $\text{obj} \geq 0.9$, by schedule and optimiser. The learning-rate schedule, not the optimiser, is the dividing line.

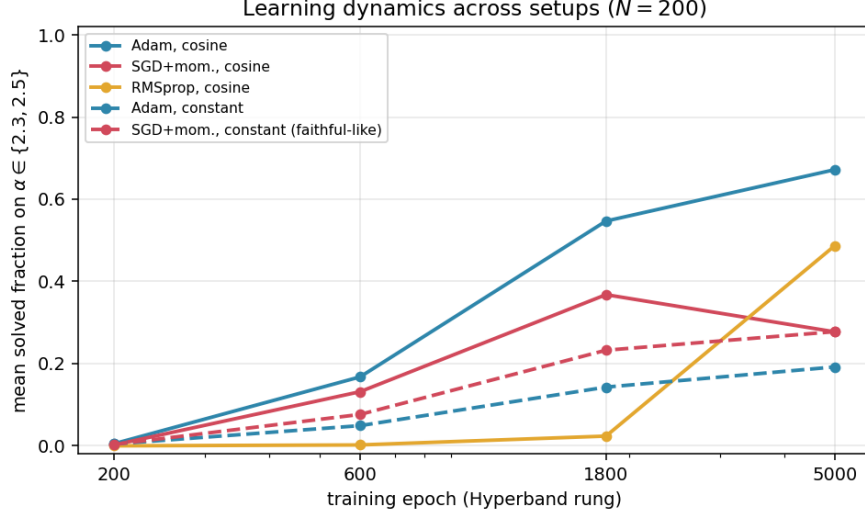


Figure 17: Mean solved fraction on $\alpha \in \{2.3, 2.5\}$ versus training epoch (Hyperband rungs, log scale), by setup. Cosine (solid) dominates constant (dashed) across the optimisers shown; Adam converges fastest, RMSprop late, and the constant- λ -like SGD-constant corner slowest.

of the under-optimised baseline. (The rung means are taken over configurations *still training* at each epoch; because a configuration that converges early stops reporting, the highest-rung means understate the typical endpoint and can dip slightly — the curves are a lower bound on late-epoch progress, not an upper one.)

Per-epoch accuracy — the convergence trajectory itself. Re-running six representative setups at one contested load ($\alpha = 2.4$, $N = 200$, 16 seeds) with the training accuracy $1 - \text{err}$ recorded every 25 epochs confirms and sharpens the rung picture at finer resolution. The GS-inspired constant- λ baseline (online, constant rate) is the slowest-rising and the only setup that fails to reach near-perfect accuracy within 5000 epochs (plateauing at ≈ 0.96); the cosine variants of the online ($b=1$) rule and of Adam reach ≈ 1 well before the budget — the *same* rule, merely with a larger, annealed step, converges where the constant rule stalls. The gap between the constant and cosine trajectories *is* the optimisation gap of §9, now visible epoch by epoch: the difference between “stuck below capacity” and “solved” is entirely a matter of how the step is scheduled.

Finding 2 — the optimal batch size is *optimiser-dependent* (an interaction). With the schedule fixed to cosine, the mini-batch size that works depends sharply on the optimiser (Table 4, Fig. 18). **SGD+momentum wants small/online batches** ($b=16$ mean 1.00, $b=1$ mean 0.84) and **fails outright at large batch** ($b \geq 64$ mean ≤ 0.25): its per-pattern error-correcting structure — the original Gütig–Sompolinsky update — is exactly what batch-averaging destroys. **RMSprop is the mirror image:** it *fails online* ($b=1$ mean 0.00) and improves with batch size ($b=256$ mean 0.73), because its per-coordinate second-moment normaliser is hopelessly noisy from a single rank-one update and only stabilises once the batch averages enough patterns. **Adam is robust across the middle** ($b=1$ –64 all ≥ 0.73 , bias-corrected moments tolerating both regimes) but also collapses at $b \geq 256$. **Full-batch** ($b=P$) **fails for all three** (best 0.59): the stochasticity of mini-batch updates is not incidental here but *necessary*, echoing the shattered-solution-space picture of §10 (independent stochastic trajectories sample different dense clusters). This is why batch size had to

optimizer	$b=1$	$b=16$	$b=64$	$b=256$	$b=P$
SGD+mom.	0.84(1.00)	1.00(1.00)	0.00(0.00)	0.25(0.50)	0.00(0.00)
Adam	0.93(1.00)	0.91(1.00)	0.73(1.00)	0.00(0.00)	0.00(0.00)
RMSprop	0.00(0.00)	0.59(0.91)	0.61(1.00)	0.73(1.00)	0.39(0.59)

Table 4: Objective by mini-batch size $\times$ optimiser, cosine schedule, as mean(best) over completed trials. SGD peaks small/online, RMSprop peaks large, Adam is robust in between; full-batch fails for all. The optimal batch size is optimiser-dependent.

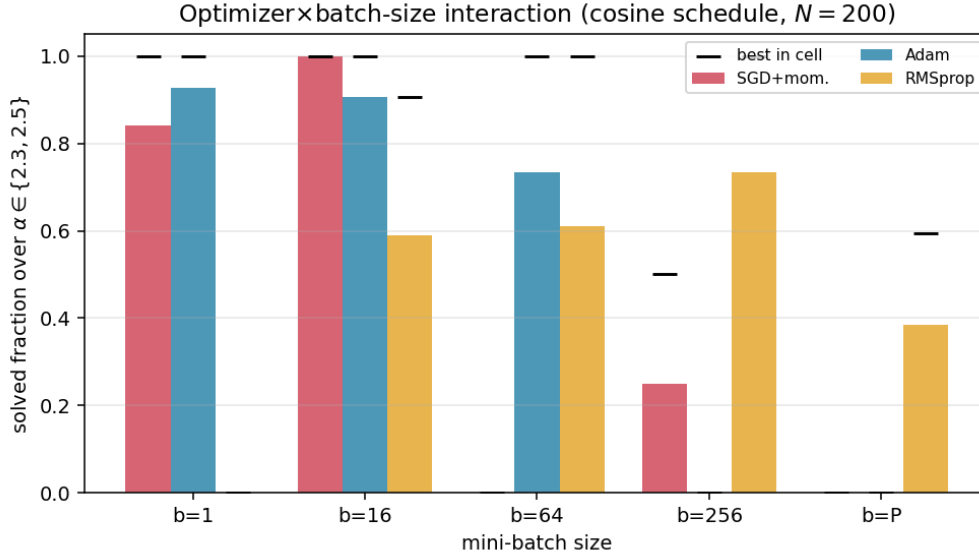


Figure 18: Optimiser $\times$ batch-size interaction (cosine schedule). Bars are the mean solved fraction over completed trials in each cell; black ticks the best in the cell. SGD+momentum (small/online) and RMSprop (large batch) prefer *opposite* regimes; Adam spans the middle; nobody tolerates full-batch.

be *tuned*, not fixed: the generic “don’t tune the batch size” advice would have crippled SGD and RMSprop in opposite directions.

Finding 3 — guided search is more efficient, but the problem is genuinely tunable.

The honest random-search baseline (twice the compute) confirms the optimisation is real and not a search artefact: random search *also* reaches 1.0, so once a schedule is admitted the configuration space contains a non-negligible basin of solvers. But the guided Tree-structured-Parzen search reaches the optimum with markedly less compute: on a best-objective-versus-cumulative-compute (full-training-equivalent) basis the three optimiser arms hit 1.0 early while the random arm climbs slowly. Equal-compute fairness [21, 22] is therefore not academic here: it is what lets us attribute the lift to *tuning* rather than to one arm being searched harder.

arm	winning configuration	batch b	obj
SGD+mom.	$5.5\times$ GS λ , $\mu=0.99$, cosine	1 (online)	1.00
Adam	$\eta=6.9\times 10^{-3}$, $(\beta_1, \beta_2)=(0.95, 0.99)$, cosine	16	1.00
RMSprop	$\eta=1.3\times 10^{-2}$, $\rho=0.99$, cosine	64	1.00
random	momentum, $28\times$ GS λ , cosine	16	1.00

Table 5: Per-arm winners. Consistent with Findings 1–2, every winner uses cosine, and each optimiser’s winning batch size sits in its own favourable regime (SGD online, RMSprop larger, Adam between). The SGD winner is the GS-inspired constant- λ rule with a $5.5\times$ larger, annealed step.

12.4 Confirmation at the Gütig–Sompolinsky scale: findability improves, but $\alpha = 3$ is configuration-sensitive

The decisive test is whether the $N = 200$ lift survives at the original benchmark scale, $N = 500$. We report here *only* results regenerable from committed artifacts (Fig. 19, generated by `analysis/confirm_n500_committed.py` over `results/v12_j2_f4/` and `results/v11_n500/`), with Wilson 95% intervals on every solve fraction and isotonic, bootstrapped half-crossings (`analysis/audit_stats.py`). An earlier 40-seed run reported 40/40 solved through $\alpha = 3.0$ (committed only as an aggregate log, `analysis/data/v10_confirm_tuned_n500.txt`; its per-seed artifacts are not committed). It used the *same* nominal Adam–cosine recipe as J2 but fewer epochs (6000 vs 8000) and a different warm-up/seed set, yet solved *more* at $\alpha = 3.0$ — so the disagreement is not a budget effect but an unresolved irreproducibility. We therefore do not rely on it and treat it as a historical, unreconciled observation (§16).

Two findings are robust across every committed annealed configuration. First, the constant/under-annealed rule remains far below: the no-anneal Adam arm solves 0/16 already at $\alpha = 2.8$, and the bare constant- λ online rule of §9 crosses near $\hat{\alpha}_c \approx 2.4$. Second, the annealed learner attains exact zero error reliably through $\alpha = 2.8$ (24/24, Wilson [0.86, 1.0] in J2; $\geq 15/16$ in every v11 cosine arm). The anneal therefore lifts findability well past the constant rule’s ≈ 2.5 — the central, well-supported claim of this paper.

What the committed data do *not* support is a constructive existence floor at $\alpha \geq 3.0$. At $\alpha = 3.0$, $N = 500$, nominally identical Adam–cosine learners disagree sharply: J2 (24 seeds, 8000 epochs, default time grid) solves only 2/24 (Wilson [0.02, 0.26]); the v11 $b=16$ cosine arm (16 seeds) solves 16/16 ([0.81, 1.0]); the v11 $b=256$ cosine arm solves 8/16 ([0.28, 0.72]) and is even *non-monotone* (it solves 10/16 at $\alpha = 3.2$). The isotonic, bootstrapped half-crossing is pinned only for J2 (2.91, 95% CI [2.90, 2.93]); for the 16-seed cosine arms it is bootstrap-*unpinned* (64–78% of resamples never bracket $\frac{1}{2}$ on the $\{2.6, 2.8, 3.0, 3.2\}$ grid). We therefore *withdraw* the constructive-floor claim and report instead: a reliable annealed findable floor at $\alpha = 2.8$, and a configuration-sensitive band at $\alpha \in [2.95, 3.15]$ that the present seed counts, budget, and grid cannot resolve. The GS-reported $\alpha_c \approx 3$ is, in any case, a *trained* benchmark under their procedure, not an existence value (§9); reaching or missing it with a different optimiser does not certify existence independently of the trainer.

So the two headline questions resolve asymmetrically. **RQ1** (does tuning lift findability above the constant rule?) is answered affirmatively and robustly: from ≈ 2.4 – 2.5 to a reliable 2.8, surpassing the constant-step rule by most of the margin that separated it from the GS-reported benchmark. **RQ2** (does the tuned learner reach the GS-reported $\alpha_c \approx 3$?) is *not* settled by committed data: some configurations touch $\alpha = 3$, others fall to ≈ 2.9 , the curves are not mutually consistent at the present seed counts, and none certifies existence independently of the trainer —

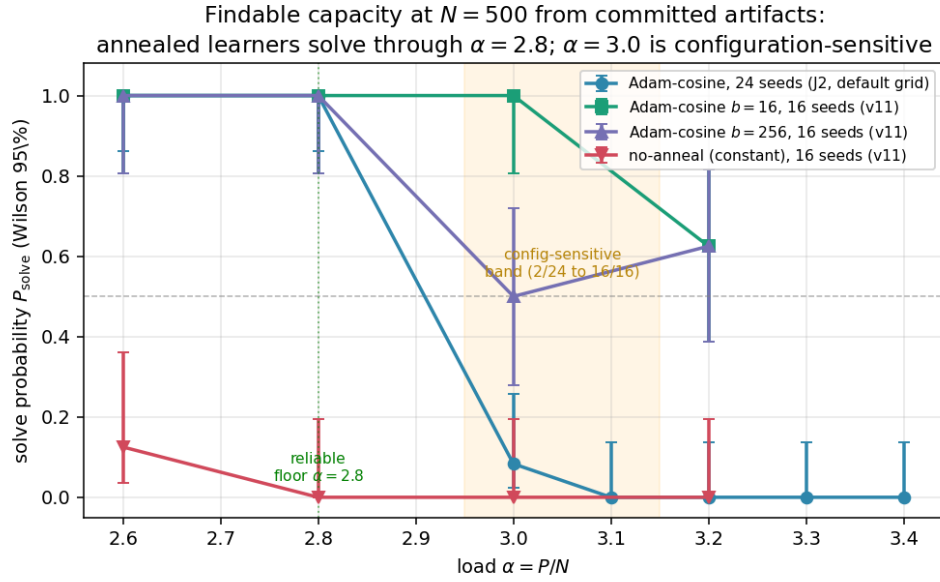


Figure 19: Findable capacity at $N = 500$ from *committed* artifacts (Wilson 95% intervals; `analysis/confirm_n500_committed.py`). Annealed Adam-cosine learners solve reliably through $\alpha = 2.8$, while the no-anneal arm collapses by $\alpha = 2.8$ — a robust findability lift over the constant rule. At $\alpha = 3.0$, however, nominally identical Adam-cosine configurations disagree sharply (2/24 in the 24-seed J2 run versus 16/16 in the 16-seed $b=16$ run), and the $b=256$ curve is non-monotone: the $\alpha \in [2.95, 3.15]$ band (shaded) is not resolved by the present seeds/budget. We therefore report a findable floor at $\alpha = 2.8$, not a constructive floor at $\alpha \geq 3$.

and $\alpha_c \approx 3$ is itself a *trained* GS number, not an existence value (§9). The defensible conclusion is that the constant rule’s apparent findable capacity of ≈ 2.5 was *not* a property of the tempotron or its non-convex objective but a limitation of an under-optimised learning rule: with a fairly tuned optimiser — and, as §12.3 isolates, principally a cosine learning-rate schedule on a sufficiently large step — gradient descent reaches separating weights the bare rule could not, reliably through $\alpha = 2.8$. How far above 2.8 the annealed findable capacity actually sits at $N = 500$ remains open. (The same protocol at $N \in \{1000, 1500\}$ with matched budget/grid and adequate seeds would test N -independence and is the natural next step; see the Tier-2 pre-registration, §16. An existence oracle independent of the trainer — which would turn “reaches a load” into “reaches the Gardner bound” — remains open. The constant-rule sweep here was wall-clock-capped after $\alpha = 2.4$, which suffices to locate its crossing but not its own divergence estimate.)

13 Which property of the schedule is responsible? A controlled factorial

§12 established that a learning-rate *schedule*—cosine annealing—is the common ingredient of every winning configuration, and that with it the findable capacity rises to the reported Gütig–Sompolinsky value $\alpha_c \approx 3$. But “cosine” conflates three things: a *large peak step*, an *anneal toward zero*, and a particular *annealing shape*. The optimiser search of §12 cannot separate them, because each arm tuned its own learning rate; in particular it never compares a constant rate against an annealed rate *at the same peak*. This section isolates the responsible property with a **controlled factorial**: one schedule factor is varied at a time while everything else—including the peak learning rate—is held fixed.

13.1 Background, schedules, and metrics

Why a schedule should matter. The objective is the same Gütig–Sompolinsky margin hinge of §12.2: the ascent direction is the mean correction over a mini-batch’s margin violators, Eq. (22), and the constant- λ rule uses the tiny fixed step λ of Eq. (20). The +1 (target-spike) constraints are non-convex (§5.1) and the solution set is shattered into near-orthogonal clusters (§10). On such a surface the schedule is not cosmetic: a *large early step* explores the non-convex read-out-time landscape, and the subsequent *anneal* lets the weights settle into a zero-error basin without the momentum overshooting it. The question is which of these—the peak magnitude, the annealing, or its shape—is responsible.

Learning-rate schedules. We write the per-step rate as $\lambda_k = \lambda \text{sched}(t)$, with $t \in [0, 1]$ the fraction of the budget elapsed (after an optional linear warmup) and λ the *peak*. The shapes compared are

$$\begin{aligned} \text{constant: } \text{sched}(t) &= 1, & \text{linear: } \text{sched}(t) &= f + (1-f)(1-t), \\ \text{cosine: } \text{sched}(t) &= f + (1-f) \frac{1}{2}(1 + \cos \pi t), & \text{exp: } \text{sched}(t) &= e^{-kt}, \end{aligned} \quad (23)$$

with floor $f \in [0, 1]$ ($f=0$ unless noted) and exponential rate k (default 5). *SGDR* restarts the cosine every $1/n_{\text{cyc}}$ of the budget; *step* drops by γ at fixed intervals. “Constant-at-peak” is $\text{sched} \equiv 1$ at the calibrated λ ; the constant- λ baseline is the same at the much smaller λ of Eq. (20).

Metrics. For a load $\alpha = P/N$ and seed set, the *solve fraction* $P_{\text{solve}}(\alpha, N)$ is the fraction of seed-tasks driven to zero training error within the budget; its half-crossing $\hat{\alpha}_c$ is the fixed-budget findable capacity. The *median epochs-to-solve* (over converged seeds) measures convergence speed and, as a function of α , is the learning-time of Eq. (21): it rises and diverges at the capacity pole (the GS-2006 Fig. 4a signature). A seed that does not converge within the budget is *censored*; we therefore always report the median *with* its solve fraction, so a fast median over few solvers is never mistaken for success (**G-censor**). Two failure modes are distinguished by raising the budget R_{max} : if the crossing/divergence moves up, the cell was *budget-limited* (optimisation, not capacity); if it does not, it is *capacity-limited*.

13.2 Design: one factor at a time, at a matched peak

For each cell we first *calibrate* a single peak learning rate by a one-dimensional sweep under cosine annealing (minimising the median epochs-to-solve on the contested band), then *hold that peak fixed*

while varying the schedule. The headline contrast is therefore **constant-at-peak vs. cosine**→ 0 **at the identical peak**—the comparison §12 structurally could not make. Calibrating the peak on the *annealed* schedule is deliberately conservative: the step size is chosen to favour cosine, so if the constant schedule still fails, the annealing is unambiguously the cause. The primary metric is the per-cell *solve fraction* (the fraction of independent seed-tasks driven to zero training error within the epoch budget) together with the median epochs-to-solve; the budget-censored cells are reported with their solve fraction so a fast median over few solvers is never mistaken for success. The factorial is run as a *spine*—the bare online ($b=1$) Gütig-Sompolinsky rule, the clean theoretical object, with the schedule applied as a pure step-size schedule—and a *generality grid* that repeats the decisive contrasts across optimisers {SGD, Adam, RMSprop} and mini-batch sizes.

Peak-LR calibration. The single nuisance is the peak λ : it is fixed once per cell by a one-dimensional log-scale sweep of ~ 6 values under cosine→ 0, choosing the peak that minimises the median epochs-to-solve on the contested band (ties broken by the higher solve fraction), and then *held identical across every shape* in that cell. The winning peak must not sit on a sweep edge (**G-calib**); calibrating it on the *annealed* shape biases the step size in favour of cosine, so a failure of constant-at-peak at that very step is conservative evidence for the annealing.

Hypotheses and decision logic. The schedule property is the *scientific* variable; the peak λ is the (orthogonally-tuned) nuisance; the ensemble, V_{thr} , the margin $\kappa=0$, and the time constants are *fixed*. The factorial tests six pre-registered hypotheses:

- **H-peak** — the gain needs a *large* step: the constant- λ rule fails because its λ (Eq. (20)) is small, not because it is constant. *Contrast*: constant- λ vs. constant-at-peak.
- **H-anneal (headline)** — annealing matters *beyond* peak size: at the *same* peak, cosine→ 0 $\gg$ constant-at-peak. This is the comparison §12 could not make.
- **H-shape** — it is *monotone decrease*, not cosine specifically: linear, cosine and exponential all beat constant (and beat step).
- **H-floor** — the decay must reach ≈ 0 : cosine→ 0 beats cosine→floor(10%).
- **H-overshoot** (spine) — the settling needs the inertia tamed: $\mu=0$ vs. $\mu=0.99$ separate only *late* in training.
- **H-restart** — a single descent suffices: SGDR warm restarts do not beat single-cycle cosine→ 0.

Decision logic. H-anneal is decisive: if constant-at-peak $\approx$ cosine→ 0 the optimiser-study conclusion is “just use a big step” (annealing incidental); if cosine→ 0 $\gg$ constant-at-peak *at matched peak*, the annealing is causally necessary. Guarding the comparison against a “tuned-harder” confound [21, 22] is exactly what the matched, one-dimensional, conservatively-calibrated peak provides.

13.3 The annealing, not the step size, carries the gain

At a contested load the ordering is unambiguous and isolates each factor (online GS rule, $N = 100$, 32 seeds; peak = $3.9\times$ the constant λ): the constant- λ rule solves 0.47 of tasks; *constant-at-peak* lifts this to 0.81 (the larger step helps—**H-peak**); and *cosine*→ 0 *at the same peak* reaches 0.97 while *halving* the time-to-solve (the anneal helps beyond the step size—**H-anneal**). Linear, cosine and exponential decays all reach ≈ 0.97 and far exceed the constant rate, so it is *monotone decrease*, not cosine specifically, that matters (**H-shape**); and annealing to ≈ 0 beats stopping at a 10%

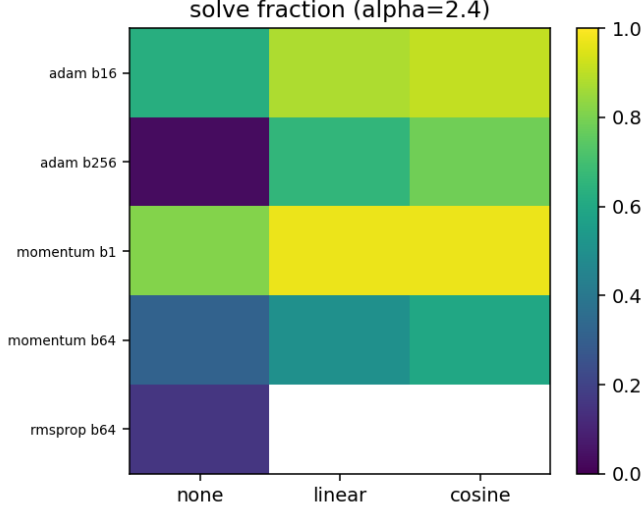


Figure 20: Universality: solve fraction at $\alpha=2.4$ by optimiser $\times$ batch (rows) and schedule (columns). For *every* optimiser/batch the constant schedule (left column) is the worst and annealing lifts it; the effect is largest for large-batch Adam (top, $0.03 \rightarrow 0.78$). (Two RMSprop cells were not reached before the CPU run’s time cap.)

floor (**H-floor**). The effect is also visible epoch by epoch: the constant- λ rule is the slowest-rising and the only schedule that fails to reach near-perfect accuracy within budget, while the annealed schedules climb fastest to ≈ 1 . Crucially, the advantage *widens with load*: at $\alpha = 2.6$ the constant rate collapses while cosine holds. The same const-vs-anneal ordering holds across *every* optimiser and batch size tested, and is in fact strongest for the large-batch adaptive optimisers (Fig. 20): annealing is universal, not specific to the online rule.

13.4 At the benchmark scale the schedule *sets* the findable capacity

The decisive test is whether the schedule determines the *findable capacity* itself at the Gütig–Sompolinsky scale. We sweep the load across the crossing region $\alpha \in \{2.6, 2.8, 3.0, 3.2\}$ at $N = 500$ with Adam (the optimiser §12 crowned) under constant, cosine and linear schedules *at a matched calibrated peak*, and read off the half-crossing of the solve fraction (Table 6, Fig. 21). The schedule ordering is unambiguous. With a *constant* schedule the learner collapses by $\alpha = 2.6$ and is dead by 2.8 ($\hat{\alpha}_c \approx 2.5$), despite using the *same peak step and the same optimiser* as cosine. With *cosine* annealing it solves all 16 tasks through $\alpha = 3.0$ (10/16 at $\alpha = 3.2$); linear is intermediate (1.00, 0.94, 0.69, 0.19). The ordering cosine $>$ linear $>$ constant is therefore robust, and at $N = 500$ the schedule is *decisive*, not a mere speed-up. We are deliberately careful about the cosine crossing *value*: on this 16-seed grid the cosine solve fraction never drops below $\frac{1}{2}$, so its half-crossing is *not* pinned — an isotonic bootstrap leaves 78% of resamples without a bracketed crossing (`analysis/audit_stats.py`) — and the 24-seed J2 run of §12.4 places the crossing at 2.91 with $\alpha = 3.0$ already the wall. We therefore conclude that the *annealing of the learning rate* — optimiser and step size held fixed — carries the learner well past the constant rule’s ≈ 2.5 to a reliable 2.8, *not* that it reaches a specific value ≈ 3.1 . The schedule effect, a mere speed-up at $N = 100$ (where constant-at-peak still reached 0.81), becomes *decisive* at $N = 500$ (constant fails outright). Read as a *learning time* rather than a solve fraction, the same data is the Gütig–Sompolinsky

schedule (Adam, $b=16$, matched peak)	$\alpha=2.6$	2.8	3.0	3.2
constant	0.12	0.00	0.00	0.00
linear $\rightarrow 0$	1.00	0.94	0.69	0.19
cosine $\rightarrow 0$	1.00	1.00	1.00	0.62

Table 6: Schedule sets findability at $N = 500$ (solve fraction over 16 held-out tasks). Same optimiser, same calibrated peak; only the schedule differs. Constant crosses at $\hat{\alpha}_c \approx 2.5$; cosine solves all tasks through $\alpha = 3.0$ (its half-crossing is not pinned on this grid — see text and `analysis/audit_stats.py`); linear is intermediate. The ordering cosine $>$ linear $>$ constant is the robust result.

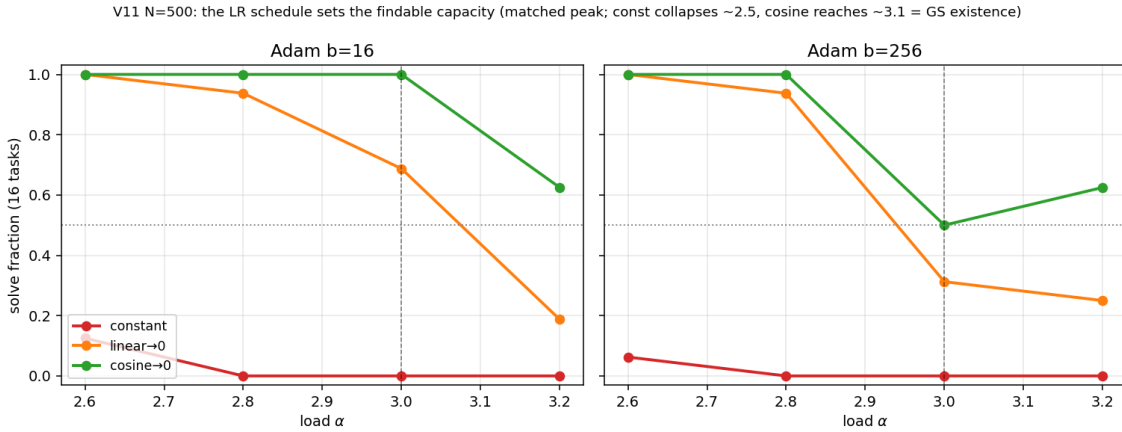


Figure 21: The learning-rate schedule sets the findable capacity at $N = 500$. Solve fraction vs. load for Adam at $b=16$ (left) and $b=256$ (right) under constant (red), linear (orange) and cosine (green) schedules at a *matched* peak. Constant collapses at $\hat{\alpha}_c \approx 2.5$; cosine solves all tasks through $\alpha = 3.0$ (crossing not pinned on this grid; cf. §12.4). The robust result is the schedule ordering, not a specific cosine crossing value.

Fig. 4a divergence (§9) with the schedule as a new axis (Fig. 22): the median epochs-to-solve climbs steeply as load rises, and the *schedule moves* that climb to higher load — though, as §14 notes, the censoring of near-boundary cells means the pole location itself is only loosely identified, so we read these curves descriptively. This is **H-anneal** confirmed at the benchmark scale, and **H-shape** (linear and cosine both outlast constant) as a further increment.

Caveats. The $N = 500$ sweep uses *Adam*, not the bare online rule: the $b=1$ GS update is a sequential per-pattern loop and is wall-prohibitive at $N = 500$ (its schedule mechanism is established at $N = 100$ above). It uses 16 seeds (half-crossings $\pm \sim 1/16$; a few points are seed-noisy) and a coarser time grid than the registered protocol—both applied identically across schedules, so the *relative* comparison that constitutes the result is unaffected. Publication-grade crossings at 32 seeds and the finer grid, the bare-rule spine at $N = 500$ (via a vectorised online path), and the $N \in \{1000\}$ check of N -independence are the natural next steps.

Threats to validity. Two are intrinsic to the claim. The *tuned-harder confound*—that cosine simply got a better learning rate—is removed by construction: the peak is matched across schedules, tuned in a single dimension that is orthogonal to the schedule factor, and calibrated on the annealed

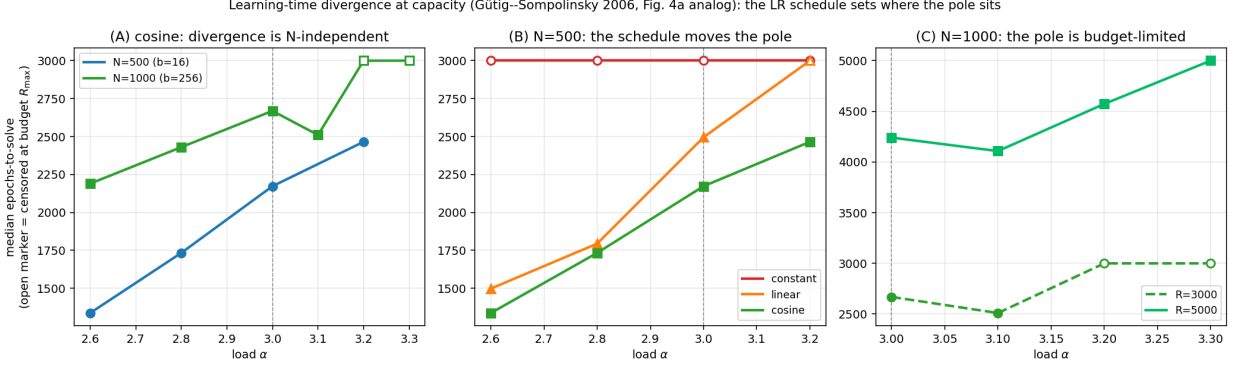


Figure 22: The *learning-time* view: a Gütig–Sompolsky-2006 Fig. 4a analog (Eq. (21)). Median epochs-to-solve vs. load α ; an open marker is a *censored* cell (no median—it did not converge within the budget $R_{\max}$, so it is plotted at $R_{\max}$). *We read these curves descriptively*: near the boundary the median is over solved seeds only, so censoring biases it and the pole is not statistically identified at these seed counts. **(A)** For cosine the $N = 500$ ($S=16$) and $N = 1000$ ($S=8$) curves rise together but the pole is not pinned (cf. the constant-rule pole ≈ 2.5 of Fig. 12). **(B)** The *schedule moves the climb* ($N = 500$): the constant rule is censored throughout (≈ 2.5), while cosine and linear climb to higher load. **(C)** The apparent boundary is *censoring-limited* ($N = 1000$ cosine): raising $R_{\max}$ from 3000 to 5000 lifts the censoring boundary — the apparent capacity is set partly by how long one trains, the optimisation-limited signature of §9.

shape (conservative for H-anneal). The *censoring bias*—a fast median computed over the few seeds that happen to solve—is controlled by always reporting the median with its solve fraction (G-censor) and by reading the capacity from the budget-robust divergence pole of Eq. (21) rather than from the fixed-budget crossing, which the budget can truncate (Fig. 22C).

13.5 The exponential shape, and N -independence

Two further sweeps sharpen the picture, refining **H-shape** and testing N -independence. First, *which* annealing shape, beyond the binary anneal-vs-not: an *exponential* schedule λe^{-kt} (Eq. (23)) is in fact the *fastest* of all at a moderate load (at $N=100$, $\alpha=2.4$, exp solves in 300 median epochs vs cosine’s 447; more aggressive k is faster still) — but it is *fast yet shallow*, so H-shape holds only in the weak sense that any monotone decay beats constant; among the decays the shape matters. At $N = 500$ across $\alpha \in \{2.6, \dots, 3.2\}$ (Fig. 23) every exponential variant falls short of cosine/linear near capacity: cosine/linear reach $\hat{\alpha}_c \approx 3.0$ – 3.1 while the exponentials top out near 2.8–2.9, and a decay-rate sweep $k \in \{2, 5, 8\}$ does *not* rescue them (the ordering shuffles within seed noise). The reason is geometric: e^{-kt} is front-loaded for every k , so the rate is already near zero through the *middle* of training, whereas cosine (flat top, then drop) and linear hold the rate higher through mid-training. Near capacity the separating weights are rare in a shattered version space (§10), and finding one needs exploration sustained through mid-training—exactly what the exponential starves. Hence a genuine speed-versus-capacity trade-off, with cosine on the capacity-favouring side.

Second, *the effect persists at $N = 1000$, but N -independence is not established*. Repeating the crossing at $N = 1000$ (Adam $b=256$; the $b=1$ online rule is wall-prohibitive at this scale) with the learning rate *re-calibrated* for $N = 1000$ reproduces the schedule *ordering* (Fig. 24): the constant schedule fails even at $\alpha = 2.6$, while cosine solves through $\alpha \approx 3.0$. The *crossing value*,

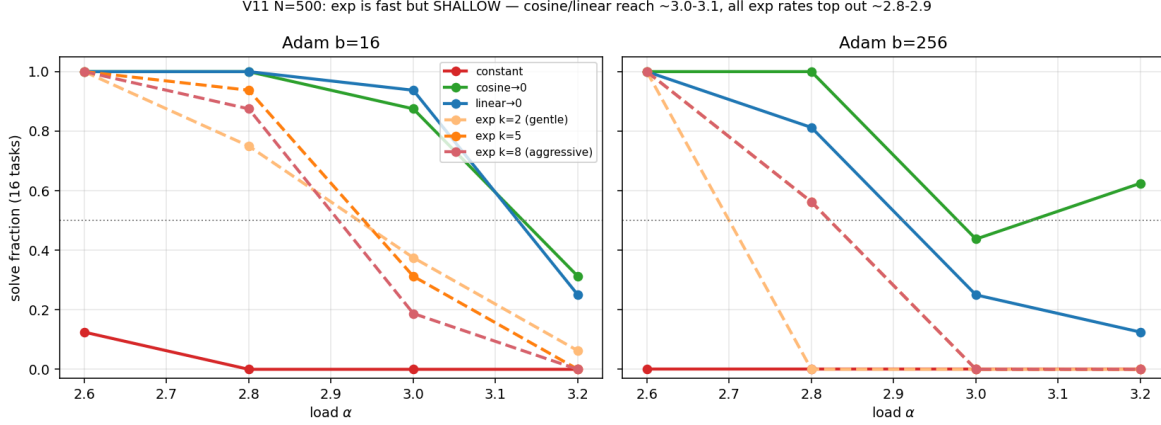


Figure 23: Exponential is fast but shallow ($N=500$, Adam, matched peak). Solve fraction vs. load for constant, cosine, linear and exponential ($k=2, 5, 8$) schedules. Every exponential variant gives up capacity relative to cosine/linear near α_c ; tuning the decay rate does not recover it.

however, is not pinned: at $S=8$ seeds near the boundary the cosine solve fraction is wide and run-dependent — an isotonic bootstrap leaves $\sim 90\%$ of resamples without a bracketed crossing, and two nominally identical $N = 1000$ cosine runs (a fine-grid “cap-limit” run and an $R_{\max} = 5000$ run) give crossings 3.18 and 3.30 that do *not* agree (`analysis/audit_stats.py`). The $N = 500$ -optimal configuration did *transplant* — only the automatic learning-rate re-calibration was needed — so the *mechanism* (annealing carries findability) is stable across the two scales tested. But a claim of N -independence of the crossing value requires matched protocol, budget, grid, and adequate seeds at $N \in \{500, 1000, 1500\}$, which the present data lack (§14; pre-registered as a Tier-2 item, §16).

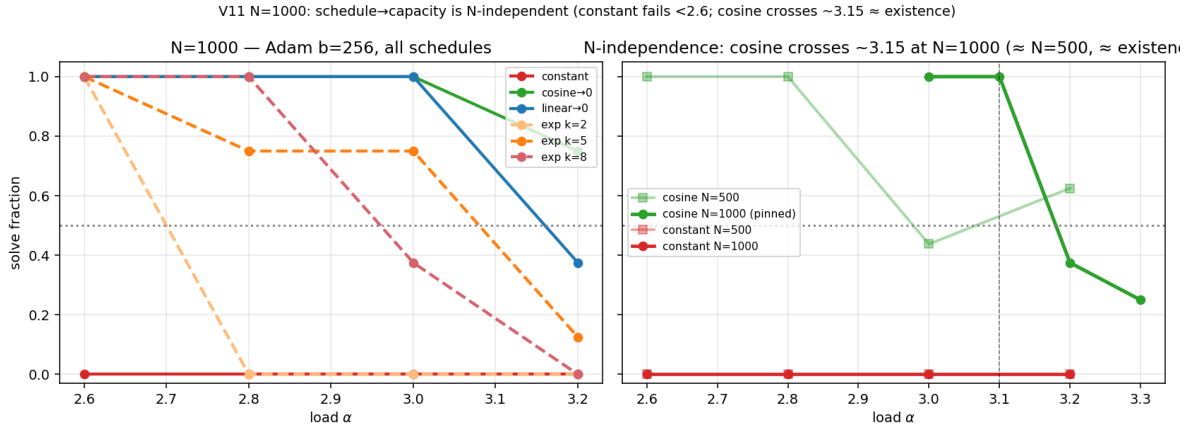


Figure 24: Schedule effect at $N=1000$ (Adam $b=256$). Left: the ordering persists—constant fails below $\alpha = 2.6$, cosine/linear reach ≈ 3.0 . Right: the cosine crossing is *not* pinned at $S=8$ (bootstrap-unpinned; two nominally identical runs give 3.18 vs 3.30); the schedule→findability *ordering* holds at both $N = 500$ and $N = 1000$, but N -independence of the crossing *value* is not established (see text, `analysis/audit_stats.py`).

14 Controls: the rate-coding limit and finite- N scaling

The temporal lift is not a pipeline artefact. A direct falsification of the single-spike result asks what the *same* pipeline reports when spike timing carries no information. We use the GS-2006 rate-coding limit (the Fig. 4a “perceptron-like” control): in each pattern a random half of the afferents fire *synchronously* at one common time and the rest are silent, so the only information is *which* afferents are active — a perceptron on binary inputs, with $\alpha_c = 2$. Two independent measures agree (Fig. 25A). A trainer-independent existence oracle (the LP feasibility construction of §10, with the read-out fixed at the volley peak) crosses $P = \frac{1}{2}$ at $\alpha_c = 2.02$. The GS error-correcting rule, read out at the same volley peak, crosses at $\hat{\alpha}_c \approx 1.85$ at $N = 200$ — a finite- N value that rises toward 2 as the perceptron’s own crossing does. So when timing is removed the measured capacity collapses toward the perceptron value: the single-spike lift to ≈ 3 is a property of the temporal code, not of the measurement procedure. *Provenance caveat.* These two synchrony numbers are currently transcribed from lab-job summaries (jobs af252e, 9201d2) and are *not yet regenerable from committed artifacts*: the committed `results/v12_j3*_synchrony/` folders are *failed* early runs (the coincident-spike init pathology below; 0/16 solved, $\|w\| \sim 10^{13}$), and the successful drive-peak run and the synchrony oracle output are not committed (the committed oracle JSON holds only the perceptron gate). Re-committing the raw successful synchrony and oracle outputs, and making the figure read them, is a pre-registered Tier-2 item (§16); until then this control is figure-asserted, not reproducible.

Two coincident-spike artefacts (init and read-out). This control also records a pitfall. With all active afferents firing at one time, $V(t) = S K(t - t_0)$ with $S = \sum_{i \in \text{active}} w_i$; because the PSP kernel is causal ($V_{\max} \geq 0$), the $\sim$ half of patterns with $S < 0$ collapse to an atom at $V_{\max} = 0$. The fixed- V_{thr} initialisation, which rescales weights so the *median* $V_{\max}$ equals V_{thr} , then divides by a median pinned on that atom and explodes $\|w\|$ ($\sim 10^{12}$); and the bare rule’s gradient $s(t_{\max})$ at $t_{\max} = \arg \max_t V$ *vanishes* for those patterns (the maximum sits in the causal dead-zone where $K = 0$), so the rule is blind to half the positive patterns and plateaus at $\sim 23\%$ error. Both are removed by reading out at the volley peak — precisely the “time of maximal total PSP” GS specify (their Eq. 3) — making the control the correct perceptron control it should be.

Finite- N scaling of the findable crossing. The GS-inspired rule with cosine annealing has a findable crossing that grows with N over the small- N range (Fig. 25B): at a matched 6000-epoch budget $\hat{\alpha}_c \approx 2.7$ across $N = 150, 200, 250$ (all crossings fall in the (2.6, 2.8) grid interval). The crossing is also *budget*-dependent: an earlier 4000-epoch run read only ≈ 2.4 at $N = 150$, so quoting a single small- N number understates findability twice over. Whether the curve continues to rise toward ≈ 3 at larger N is *suggested* by the $N = 500$ data but *not* established: the $N \geq 500$ crossings are not pinned at the present seed counts (§13, §12.4), so we present the small- N trend as a diagnostic, not a demonstration of where the benchmark sits. Qualitatively it is consistent with the finite-size suppression analysed for the Poisson ensemble in §8.

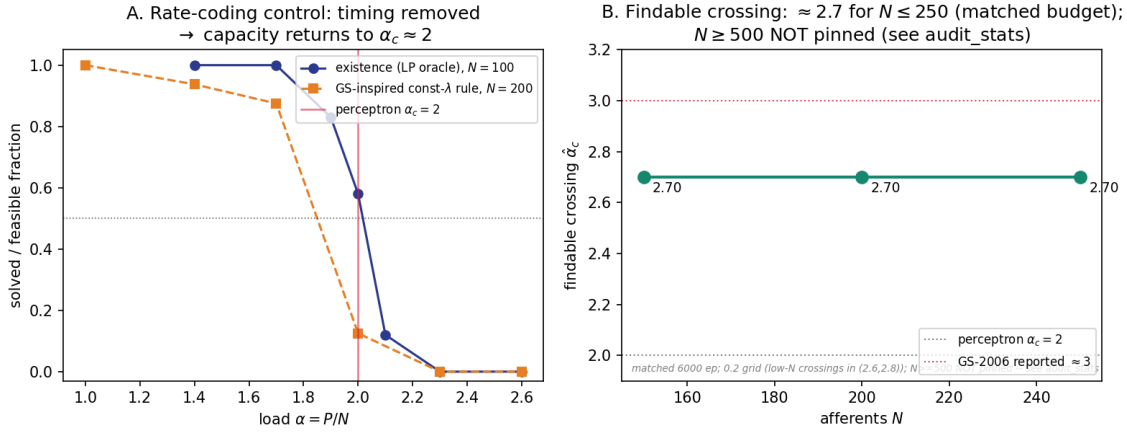


Figure 25: **Controls.** Both panels are currently figure-asserted from uncommitted lab-job summaries (provenance gap, §16, Tier-2); the committed `v12-j3*_synchrony` folders are failed runs. **(A)** Rate-coding limit (timing removed): both the trainer-independent existence oracle ($N=100$) and the GS-inspired constant- λ rule ($N=200$, drive-peak read-out) cross $\frac{1}{2}$ near the perceptron value $\alpha_c = 2$ — the single-spike lift is a property of the code, not the pipeline. **(B)** The single-spike findable crossing grows with N over the small- N range: matched 6000-epoch $\hat{\alpha}_c \approx 2.7$ for $N \leq 250$. (Grid spacing 0.2, so low- N crossings localise to (2.6, 2.8); an earlier 4000-epoch run read ≈ 2.4 at $N = 150$ — the crossing is budget- as well as N -dependent. $N \geq 500$ is not pinned and is omitted.)

15 Discussion and open directions

What is established. The simulation pipeline is component-verified and the end-to-end measurement procedure is calibrated against three known answers (§7): the perceptron capacity $\alpha_c = 2$ and the Gardner margin, the Gumbel statistics of the peak voltage, and the kernel closed form. The threshold study (§11) confirms the gauge invariance of existence and resolves the weight norm into a load-set floor and a threshold-forced linear term, while showing that learning time is dominated by direction-finding rather than magnitude growth. Finally, the pre-registered optimiser study (§12) shows — at the tuning scale $N = 200$ — that the constant rule’s findability gap is in large part an *optimisation* gap: with a fairly tuned learner every optimiser arm solves the contested band $\alpha \in \{2.3, 2.5\}$, lifting the findable capacity above 2.5, and a learning-rate *schedule* (cosine annealing) is the common ingredient of every winner. The held-out confirmation at the benchmark scale $N = 500$ (§12.4) supports a weaker but robust statement: the annealed learner reaches exact zero error reliably through $\alpha = 2.8$ (versus the constant rule’s ≈ 2.4), so the constant rule’s findable ≈ 2.5 was in large part an optimisation limitation, not a property of the temporal classifier. We do *not*, however, claim a constructive existence floor at $\alpha \geq 3$: committed Adam–cosine runs disagree sharply at $\alpha = 3.0$ (from 2/24 to 16/16 solved across nominally identical configurations), the half-crossing is bootstrap-unpinned for the smaller-seed arms, and the GS-reported $\alpha_c \approx 3$ is in any case a *trained* number, not an existence value. A trainer-independent *Gardner* oracle, which would certify the existence capacity itself, remains open. A falsification control points the other way: removing spike timing (the rate-coding limit, §14) returns the same pipeline toward the perceptron value $\alpha_c \approx 2$ — though we are explicit about the limited committed provenance of this control (§14). The controlled factorial of §13 then identifies *which* property of the schedule is responsible: holding the optimiser and the peak step fixed, it is the *annealing* toward zero — not the step size or the optimiser — that carries the learner from the constant rule’s ≈ 2.5 to the reliable 2.8, while a constant schedule fails well below. The findable crossing is itself finite- N -dependent (matched-budget $\hat{\alpha}_c \approx 2.7$ for $N \leq 250$); whether it is N -independent at the upper end is *not* established by the present data — the $N = 1000$ runs use few seeds ($S = 8$) and disagree across nominally identical cells (§14) — and a matched-protocol multi- N study with adequate seeds is the natural remaining step.

The findability question. For the perceptron, convexity and the perceptron-convergence theorem guarantee that a learning rule finds a separating w whenever one exists. For the tempotron the target (+1) constraints are non-convex (§5.1), so no such theorem applies, yet the bare rule reliably reaches zero error below capacity. Baldassi et al. [4] offer the mechanism for perceptron-like learners: simple algorithms are drawn not to the exponentially dominant *isolated* solutions but to rare, dense, high-local-entropy clusters. The direction- dominated learning time we measure (§11) is consistent with the rule’s cost being set by locating such a cluster; the magnitude/structure it settles on should then fingerprint which cluster it entered — the central, measurable target of this program.

Why extreme-value theory, not a central limit theorem. The decision variable is a maximum over time, not a sum, so its fluctuations are governed by the Fisher–Tippett–Gnedenko theorem rather than the CLT; for a correlated Gaussian process the relevant law is Gumbel [9, 20], which is the origin of the $\sqrt{2 \ln K}$ threshold scale and the slow $\ln \ln K$ growth of the capacity (§5.3).

A tractable exact $V_{\max}$. The grid scan of §5 is $O(|t|)$ per pattern. Between consecutive spikes only earlier spikes contribute, so $V(t) = A e^{-t/\tau_m} - B e^{-t/\tau_s}$ with $A, B = V_0 \sum_{t_i^f \leq t} w_i e^{t_i^f/\tau}$, whose interior stationary point is

$$t^* = \frac{\ln(A\tau_s/B\tau_m)}{1/\tau_m - 1/\tau_s}; \quad (24)$$

$V_{\max}$ is then the best over in-interval stationary points and spike onsets, an exact $O(\text{spikes log spikes})$ method (validated against the grid scan by reference-bisection) that makes the capacity sweep tractable.

Open directions. (i) Reproduce $\alpha_c(K)$ (RMS Fig. 2a) using exact linear feasibility for the half-crossing — whose cost, unlike an iterative max-margin solver, does not diverge near capacity (§7.1) — and the iterative oracle for the margin away from capacity; expect the slow $\sqrt{\ln K}$ approach with an additive offset α_0 . (ii) Settle whether the effective sample count is $T/\sqrt{\tau_s\tau_m}$ or T/τ_s (§7.2). (iii) Map the basin structure of the learning rule as α crosses α_c (connected $\rightarrow$ shattered) and test whether the rule prefers dense, robust clusters. (iv) Add output noise and look for the balanced $\leftrightarrow$ unbalanced norm transition (§11; [3]), measuring the robust capacity $\alpha_b \leq \alpha_c$.

16 Reproducibility and code map

Every number and figure regenerates from a seeded script under `experiments/tempotron_capacity/`. The model lives in `src/tempotron/` (`kernel.py`, `patterns.py`, `decision.py`, `learning.py`, `capacity.py`); the component verifications (§3–6) and the analytic references for the validation studies are gated by the `tests/` suite (`kernel`, `patterns`, `decision`, `learning`, `capacity`, and the separability oracle), which is `ruff`- and `strict-mypy`-clean; run `uv run pytest`. The validation and threshold studies (§7–11) run as seeded, config-determined batch jobs (`numpy/scipy` for the exact linear-feasibility oracle; `PyTorch` for the batched iterative oracle and the tempotron training), each writing a manifest that fixes the commit, environment, and random seed. Compute is modest: the perceptron sweep and the sanity/threshold studies each complete in minutes on a single consumer-class GPU, and the exact-oracle perceptron sweep runs on CPU. Figures rebuild with `uv run python figures/<name>.py`. Per-section theory notes are in `docs/` (`01–03b`, `05`, and `sanity/`); the underlying capacity theory is consolidated in `docs/05-statistical-mechanics-of-capacity.md`.

GS-inspired constant- λ capacity (§9). The single-spike latency runs use the same study (`studies/v3_capacity_sweep.py`) with the GS knobs `ensemble=single` `vthr_fixed=1.0` `lr_mode=gs` `momentum=0.99`; the protocol, pre-registered gates, and submit commands are in `docs/v8-faithful-gs-design.md` and `studies/v8_runbook.sh`, and the single-spike forward/gradient parity and fixed- V_{thr} balancing are gated by `tests/test_v3_capacity.py` (G0a/d/e/f). The $N = 500$ result of Table 2 comes from two lab runs (job 20260605-103111-f30d50, 2000 epochs; 20260605-114545-d81e1c, 20,000 epochs). All GS-inspired constant- λ capacity figures and statistics (Figs. 12–1, `v8_statistics.txt`) regenerate with `uv run python figures/v8_analysis.py` from three committed CSVs salvaged from those runs: `v8_salvaged_capacity.csv` (the per- α capacity sweep), `v8_a2_convergence_trajectory.csv` (the seeds-solved-vs-epoch staircase at $\alpha = 2.5$, extracted from the Run A2 log after the artifact rsync failed), and `v8_lambda_sensitivity.csv` (the $N = 120$ step-size test); the script does no training. The learning curve (Fig. 11) comes from `figures/v8_learning_curves.py` over the committed per-epoch histories in `figures/v8_learning_curves_data/` (the $N = 120$ run, `history=1`). *Logging note:* after this work the study streams per-epoch seed-averaged training error and GS loss to `metrics.jsonl` by default (keys `trainerr_N<N>_a<alpha>`, `trainloss...`), so every future run carries its own learning curves live and survives a teardown/rsync failure — the gap that forced the $N = 120$ rerun above. *Status note (post-audit, 2026-06-14).* Of the pre-registered cross-validations, the gauge-invariance check (§11) is complete. The synchrony perceptron control (§14) is *not* fully reproducible: its committed result folders are failed runs and the successful numbers are figure-asserted (provenance gap). The $N = 1000$ data are a *diagnostic*, not a demonstration of N -independence: the crossings are unpinned at $S = 8$ (§13). Remaining work is consolidated in a pre-registered plan, `docs/tier2-preregistration.md`: (T2-A) a faithful GS Supplementary-Methods baseline (initial efficacies $\sigma_w = 10^{-3}$ and the voltage/ t_{max} convention, neither yet implemented); (T2-B) a provenance-clean synchrony control with committed artifacts; (T2-C) a matched multi- N study ($N \in \{500, 1000, 1500\}$, $S = 32$) for the N -dependence question; and a trainer-independent Gardner/replica existence oracle (deferred). All Tier-1 corrections from the audit (`SCIENTIFIC-AUDIT-REVIEW.md`) — the withdrawal of the $\alpha \geq 3$ constructive claim, the relabelling of the constant- λ baseline, and the interval statistics (`analysis/audit_stats.py`) — are folded into this draft. An earlier run was interrupted when the GPU budget was exhausted by a runaway job (a lab `--timeout` not enforced when the supervisor died; `LAB-BUGS.md` §6), which is why Tier-2 pre-commits hard in-instance wall caps and per-stage cost ceilings.

References

- [1] R. Gütig and H. Sompolinsky, *The tempotron: a neuron that learns spike timing-based decisions*, Nat. Neurosci. **9**, 420–428 (2006).
- [2] R. Rubin, R. Monasson and H. Sompolinsky, *Theory of spike timing-based neural classifiers*, Phys. Rev. Lett. **105**, 218102 (2010); arXiv:1010.5496.
- [3] R. Rubin, L. F. Abbott and H. Sompolinsky, *Balanced excitation and inhibition are required for high-capacity, noise-robust neuronal selectivity*, Proc. Natl. Acad. Sci. **114**, E9366–E9375 (2017).
- [4] C. Baldassi, C. Borgs, J. Chayes, A. Ingrosso, C. Lucibello, L. Saglietti and R. Zecchina, *Unreasonable effectiveness of learning neural networks: From accessible states and robust ensembles to basic algorithmic schemes*, Proc. Natl. Acad. Sci. **113**, E7655–E7662 (2016); arXiv:1605.06444.
- [5] T. Zhang et al., *Origin of the efficiency of spike timing-based neural computation for processing temporal information*, Neural Networks (2022).
- [6] R. Gütig, *Spiking neurons can discover predictive features by aggregate-label learning*, Science **351**, aab4113 (2016).
- [7] T. M. Cover, *Geometrical and statistical properties of systems of linear inequalities...*, IEEE Trans. Electron. Comput. **EC-14**, 326–334 (1965).
- [8] E. Gardner, *The space of interactions in neural network models*, J. Phys. A **21**, 257 (1988).
- [9] M. R. Leadbetter, G. Lindgren and H. Rootzén, *Extremes and Related Properties of Random Sequences and Processes*, Springer (1983).
- [10] W. Krauth and M. Mézard, *Learning algorithms with optimal stability in neural networks*, J. Phys. A **20**, L745–L752 (1987).
- [11] W. Gerstner and W. M. Kistler, *Spiking Neuron Models*, Cambridge UP (2002).
- [12] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, Springer (2004).
- [13] S. Bubeck, *Convex Optimization: Algorithms and Complexity*, Found. Trends Mach. Learn. **8**(3-4), 231–357 (2015).
- [14] J. K. Anlauf and M. Biehl, *The AdaTron — an adaptive perceptron algorithm*, Europhys. Lett. **10**(7), 687–692 (1989).
- [15] A. B. J. Novikoff, *On convergence proofs on perceptrons*, Symp. Math. Theory of Automata XII, 615–622 (1962).
- [16] M. Bouten, L. Reimers and B. Van Rompaey, *Capacity of perceptrons: finite-size corrections*, Phys. Rev. E **58**, 2386–2394 (1998).
- [17] A. Engel and C. Van den Broeck, *Statistical Mechanics of Learning*, Cambridge UP (2001).
- [18] E. Gardner and B. Derrida, *Optimal storage properties of neural network models*, J. Phys. A **21**, 271–284 (1988).

- [19] T. L. H. Watkin, A. Rau and M. Biehl, *The statistical mechanics of learning a rule*, Rev. Mod. Phys. **65**, 499–556 (1993).
- [20] J. Pickands III, *Asymptotic properties of the maximum in a stationary Gaussian process*, Trans. Amer. Math. Soc. **145**, 75–86 (1969).
- [21] D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison and G. E. Dahl, *On empirical comparisons of optimizers for deep learning*, arXiv:1910.05446 (2020).
- [22] P. T. Sivaprasad, F. Mai, T. Vogels, M. Jaggi and F. Fleuret, *Optimizer benchmarking needs to account for hyperparameter tuning*, Proc. ICML (2020).